claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_d2c10282-601\agent-af5800ff62c0b9c44.jsonl`
- SHA-256: `8c8a20ee4b65027a07d710565be0fc0b0d184b4f4d57bfb58893cce3e2cb3e18`
- Source modified: `2026-07-04T11:55:06+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d2c10282-601`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`

Transcript

1. user (2026-07-04T11:48:25.938Z)

```
You are drafting ONE section of the A13 "alpha launch review" for the KhelSutra badminton-highlight alpha
(repo Khelsutra/badminton-highlight-indexer, engine checkout at
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer; SPA/site repo at
C:/Users/avidu/Projects/khelsutra-guru/khelsutra).
Context: the A1 serving stack is LIVE on Cloud Run asia-southeast1 ? control plane service
'rally-control-plane' (rev 00007, edge-auth ON via Cloudflare Access at api.khelsutra.guru, 4-email
allowlist, maxScale=1, per-instance ephemeral SQLite state), L4 GPU Job 'rally-worker' (WASB-only alpha:
indexing.skip_ai_handoff=true ? no Gemini in the serving config), storage gs://khelsutra-rally-serving.
Deploy tooling: deploy/cloudrun/gen_service_yaml.py + README.md + Dockerfile; e2e evidence:
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md (READ IT).
Tracker: docs/ALPHA_LAUNCH_READINESS.md (§7 rows; A10/A11 ship-gate + WASB-C3 provenance are OWNER-GATED
and still open ? factor that into anything launch-facing).
READ-ONLY: do not modify any file, do not run git writes, do not touch GCP. Windows box, PowerShell tool.
Ground EVERY claim in a file:line or issue number you actually opened ? no invented provenance. If you
can't verify something, mark it [unverified]. Your output is a MARKDOWN SECTION consumed by an orchestrator
assembling the review doc ? return it via the schema, self-contained, concise, evidence-cited.

SECTION: **Known limits + rollback/disable steps**. Compile:
- Known-limits table from: open issues (gh issue list --repo Khelsutra/badminton-highlight-indexer --state open
  ? read the relevant ones: #471 ephemeral state, #472 cert, #443 CORS, #332 NSFW, #327 path-jail schemes,
  #301 retention, #328/#326 observability, #331/#349 perf, #175, #172), the e2e runlog §5 notes (upload
  resolution floor >=1280x720, services-replace no-op, 422-before-401 quirk), locked alpha decisions
  (WASB-only / skip_ai_handoff=true; GPU quota=1 region asia-southeast1; maxScale=1; upload size cap ?
  find the actual configured MB limit in backend config/models or uploads code and cite it).
- WASB-C3 provenance (A11, GATED): weights not commercially cleared ? what that means for who may use
  the alpha and what copy must NOT claim. Cite the Dockerfile warning + tracker row.
- A10 ship-gate (GATED): quality target not owner-ratified (#172) ? the alpha cannot make quality claims.
```

- Rollback/disable steps inventory, each with the exact command: disable the whole alpha (route/CF app off
  or scale CP to 0), roll back to a previous image digest, revoke a tester, emergency stop of a running L4
  execution, delete user data for a given video_id (bucket paths + DB reality).

2. user (2026-07-04T11:48:39.632Z)

__pycache__
artifacts
backend
backend.db
CLAUDE.md
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
README.md
requirements.txt
requirements-trackers.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
SESSION_HANDOFF.md
tests
tools
training
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
README.md

3. user (2026-07-04T11:48:39.683Z)

A1 alpha serving ? post-merge e2e deploy test (cloud-serving)

Date: 2026-07-04 · **Region:** asia-southeast1 · **Project:** gen-lang-client-0306026826
Scope: A1 (ALPHA_LAUNCH_READINESS §7) closure ? prove the merged serving fixes
(#465 input jail, #467 upload staging, #468 drain concurrency, #469 signed-URL download,
#470 auth-in-base-image) plus the committed manifest generator (#474) on a REAL deployed build,
then restore the live edge-auth posture.

> Companion to `DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md` (PR #464): that report stood the
> stack up and FILED #461/#463/#466; THIS report proves their fixes end-to-end on merged master and
> records the one new bug the run surfaced (#475, fixed by #476 mid-run).

1. What was deployed

Resource	Value
Source	merged `master` ? `195932f` (first build) ? **`3c97117`** (final build, incl. #476)

Worker image	`?/rally/rally-worker:latest` ? build 1 `sha256:ed2fe448?` (195932f), build 2 **`sha256:31475f27?`** (3c97117); the Job re-pinned via `gcloud run jobs update` after each build
L4 worker **Job**	`rally-worker` ? unchanged shape (nvidia-l4, 8 vCPU/32Gi, FUSE `khelsutra-rally-serving`?`/gcs`)
Control-plane **Service**	`rally-control-plane` ? rev **00005** (auth-off, build 1) ? rev **00006** (auth-off, final digest) ? rev **00007** (auth-ON restore, final digest). Deployed FROM THE BASE IMAGE directly ? no derived PyJWT overlay (**#470 proven**); `maxScale=1` (#471 option 1)
Manifest	generated by the **committed** `deploy/cloudrun/gen_service_yaml.py` (#474) ? `--edge-auth off` for the test window, `--edge-auth on` to restore. The scratchpad-artifact gap (#473) is closed: this deploy used only repo-committed tooling

2. Boot / health (rev 00005+, identity-token auth)

- `GET /api/health` ? `200 {"status":"ok",?}`; `GET /api/capabilities` ? `wasb_hybrid` present,
  `gemini_key_present=false` (skip_ai_handoff honored), ffmpeg/ffprobe present.
- Before-state captured on the outgoing rev 00004: `/api/process` without a CF JWT ? **401**
  `edge auth: missing Cf-Access-Jwt-Assertion header` ? the M9 posture was live as documented.

3. CUJ on the FINAL image (rev 00006, digest `31475f27?`) ? all green

1. **gs:// process** ? `POST /api/process {gs://?/MahadevpuraSingles.MP4, wasb_hybrid,
   compute=cloud}` ? 202 ? `validating ? dispatching (cloud_run) ? done` ? **48 play segments**
   (same count as the #464 run; job `18dc6502?`).
2. **#461 upload staging** ? chunked upload (`init ? part/0 ? complete`) of a 105,345-byte 720p
   clip ? `POST /api/process` with the returned **local** path + `compute=cloud` ? the job staged
   it to `gs://khelsutra-rally-serving/videos/d8335f51?/upload-e2e-720p.mp4` (byte-exact size
   verified in the bucket) and dispatched to the L4 ? **success** (0 segments ? correct for a
   synthetic test pattern). NOT the pre-fix "cloud-serving needs a cloud input" hard-fail.
   Verified on BOTH builds (`ed2fe448?` and the final `31475f27?`).
   - ? Upload validation floor: a 640×360 test clip is rejected (`resolution_fps_check`,
     min 1280×720) ? the first attempt failed on this; alpha docs should mention the floor.
3. **#468 drain concurrency, observed live** ? the upload job staged + dispatched **while** the
   48-segment reel compile was stitching on the CP: remote dispatch overlapped the local compile
   lane exactly as designed (no serialization, no false-empty stall). On rev 00006 the two process
   jobs (gs:// + upload) also ran the drain concurrently.
4. **compile** ? `POST /api/compile {video_id, segment_ids:[1..48], "e2e.mp4"}` ? 202 ? done;
   measured stitch wall on rev 00005: created 10:39:32 ? finished 10:44:26 (**~4m54s**, incl. the
   gs:// source re-fetch); rev 00006 re-run equally green. Reel **mirrored to
   `gs://khelsutra-rally-serving/highlights/915473?/e2e.mp4` ? 174,046,228 bytes (~166 MiB)**.
5. **#463 signed-URL download ? VERIFIED**:
   - `GET /api/highlights/915473?/e2e.mp4` ? **`307`** ?
     `https://storage.googleapis.com/khelsutra-rally-serving/highlights/?/e2e.mp4?X-Goog-Algorithm=GOOG4-RSA-SHA256&X-Goog-Credential=492532266377-compute@?&X-Goog-Expires=3600&X-Goog-Signature=?`
     (v4, signed AS the runtime SA via keyless signBlob ? no key file anywhere).
   - `curl -L` through the redirect: **full 174,046,228 bytes in 8.1s** (single GET ? no 32 MiB

edge-500, no Range workaround), and the file **ffprobes valid**: H.264 1920×1080 @ 29.97 fps
+ AAC, duration 244.29s (the 48-rally reel).

4. Bug surfaced mid-run: #475 ? fixed by #476 (merged, redeployed)

On the FIRST attempt (rev 00005, build `ed2fe448?` = pre-#476), the #463 check **failed**:
`GET /api/highlights/...` ? **500**; log:
`[highlights] signed-url path failed ?: you need a private key to sign credentials`.

- The reel WAS mirrored correctly; the **keyless signBlob path** failed and the handler fell back
  to serving the 166 MiB file locally ? Cloud Run's ~32 MiB cap ? edge-500 (the original #463
  symptom, resurrected by a different root cause).
- **Root cause (verified live):** `GCSStorage.signed_url()` reused the storage client's own token
  for `iamcredentials.signBlob`; `google.cloud.storage.Client.SCOPE` is **devstorage-only**, which
  signBlob rejects (`ACCESS_TOKEN_SCOPE_INSUFFICIENT`) ? and the branch's bare `except: pass`
  swallowed the real error, so the fallthrough raised the misleading private-key message.
  Environment was clean: the SA holds self-`serviceAccountTokenCreator` and
  `iamcredentials.googleapis.com` is enabled ? code bug, not config.
- **Why tests never saw it:** the fakes carry no `_credentials`; this was the FIRST run of #469's
  path on real compute credentials. Filed **#475** ? fixed by **#476** (`_signblob_identity()`
  mints a fresh `cloud-platform`-scoped ADC credential; failures now LOGGED before falling back;
  regression tests reproduce the credential shape). Suite 1659? / cov 85.46% / ruff+mypy clean;
  merged on green CI per the standing agreement; rebuilt (`31475f27?`) and redeployed (rev 00006);
  §3.5 is the pass on the fixed build.

5. Notes / observations (rough edges captured)

- **`services replace` with an unchanged manifest is a NO-OP** ? it does not re-resolve `:latest`
  after a rebuild. Force a new revision (`gcloud run services update --image ?:latest`) or stamp a
  revision suffix. Follow-up candidate: `--revision-suffix` in `gen_service_yaml.py`.
- **A redeploy wipes the per-instance SQLite state** ? `/api/highlights` then 404s on the DB
  record check (`backend/main.py:1052`) even though the reel mirror exists in gs://. Known, filed
  as **#471**; the CUJ was re-run on rev 00006 to regenerate state. Alpha-operator implication: a
  deploy invalidates in-flight jobs and past download URLs (re-process regenerates them).
- Upload validation floor (?1280×720) ? §3.2.
- 422-before-401 quirk: a **malformed-JSON** body gets FastAPI's 422 before the edge-auth
  dependency runs; any well-formed request without a CF JWT still 401s. No exposure; noted so a
  422 in edge-auth triage isn't misread.

6. Edge-auth restore + live posture (rev 00007, digest `31475f27?`)

- Redeployed with the generator's `--edge-auth on` manifest ? rev **00007**.
- `GET /api/health` ? **200** (deliberately exempt).
- `POST /api/process` (valid JSON, no CF JWT) ? **401**

`edge auth: missing Cf-Access-Jwt-Assertion header`.
- `https://api.khelsutra.guru/api/health` (no CF session) ? **302** ?
`https://khelsutra.cloudflareaccess.com/cdn-cgi/access/login/api.khelsutra.guru??` with the
Access-app aud `c980da67?082b` in the login meta ? the exact pre-test posture, restored.

7. A1 DoD checklist

DoD item	Status
Serving endpoint deployed + healthy	? rev 00007, final digest `31475f27?`
Upload ? process ? jobs ? compile ? download CUJ	? end-to-end on the final image
Edge-auth (M9) restored + verified	? 401 sans JWT; api.khelsutra.guru ? CF login 302
>32 MiB download via signed URL (#463)	? 307 + full 174 MB single-GET + ffprobe-valid
Browser-upload staging to gs:// (#461)	? staged byte-exact + dispatched + success
Parallel drain (#466/#468) observed live	? dispatch overlapped compile; 2 process jobs drained concurrently

A1 ? DONE (flipped in `docs/ALPHA_LAUNCH_READINESS.md` §7 in this PR).

4. user (2026-07-04T11:48:46.099Z)

Alpha Launch Readiness Plan

> **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu; **Created:** 2026-07-03; **Last updated:** 2026-07-04
> **Lifecycle:** `DRAFT -> IN PROGRESS -> DONE -> archived`
> **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed in `docs/README.md` and `docs/DOC_STATUS.md`.
> **Relation to existing docs:** consolidates the launch-relevant pending items from `NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`, `R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `khelsutra` Studio docs.
> **Honesty note:** `[verified]` means checked against the local synced repos on 2026-07-03. `[analysis]` means derived from a local command run on that snapshot. This is not legal, financial, or launch approval.

---

0. TL;DR

The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass. `[verified]` The golden manifest has 15 badminton videos; `[analysis]` A2-A9 made the local manifest/path, full-fusion/audio, decision, and shadow-training surfaces explicit; `[analysis]` A4/A5 refreshed the n=15 nightly and heuristic floors; `[analysis]` A8/A9 still reject served Gate A default promotion and owned-model serving claims on the current evidence.

The first controlled alpha should be a hosted, authenticated product loop: upload a video, process it, select rallies, compile, and download a reel. The owned-model/commercial rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the WASB checkpoint provenance is resolved.

1. Problem & goal

The previous launch blockers were spread across roadmap docs and mixed together three different questions:

- **Can an invited tester use the product without SSH?**
- **Can the 15-video golden corpus now support honest quality gates?**
- **Can we claim or ship an owned/commercial rally detector?**

This doc separates those into executable workstreams. The goal is to reach an invite-only alpha where the product is usable end-to-end, the quality claims are honest and reproducible, and the remaining model/legal gates are explicit rather than accidentally implied by launch activity.

2. Decisions locked

#	Decision	Source / date	Implication
D1	**Product alpha and owned-model launch are separate decisions.**	`NEXT_STEPS.md` NO-GO banner, 2026-06-30; verified 2026-07-03	We can launch a controlled hosted product loop without claiming the owned model is commercially cleared.
D2	**First alpha must be page-driven, not SSH/CLI-driven.**	`NEXT_STEPS.md` P0 UI-driven/shareable item; verified 2026-07-03	Hosted service, edge auth, upload, job polling, compile, and download are launch-critical.
D3	**The 15-video corpus is now a measurement asset, not a free launch pass.**	`[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local eval runs, 2026-07-03	Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims.
D4	**Do not promote served Gate A by default from the current n=15 evidence.**	`[analysis]` `backend.eval.serve_contrast` run, 2026-07-04	Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification.
D5	**P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.**	sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`; verified 2026-07-03	First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable.
D6	**Gen-0 n=15 shadow training is non-serving.**	`[analysis]` `training.gen0.harness` run, 2026-07-04	Learned mean beats the heuristic floor, but the paired sign test is not significant; #172 and WASB C3 remain hard blockers.

3. Verified snapshot, 2026-07-03

All five Git checkouts were fast-forwarded before analysis:

Repo	Branch	Synced HEAD
`badminton-highlight-indexer`	`master`	`e4914d1`
`khelsutra`	`master`	`b0f70a5`
`rally-annotator`	`main`	`ef9e088`
`sports-data-collector`	`master`	`bfc014f`
`sports-obsreport`	`main`	`a0c7252`

Corpus/eval facts:

- `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says `output/human_lovo_manifest.json` is the ingested golden corpus at `n = 15`.
- `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest found all 15 trajectories and labels in this workspace.
- `[analysis]` A7 generated full shuttle/person fusion schema for all 15 videos and A8 audio augmentation populated audio features for all 15.
- `[analysis]` `python -m backend.eval.fusion_compare --features-dir output` now runs over all 15 and measures person-fusion `Delta F1 = +0.008`; the paired CI crosses zero, so person fusion is not promotable by itself.
- `[analysis]` `python -m backend.eval.fusion_audio --features-dir output --compare` measures audio `Delta F1 = +0.018`; the paired CI crosses zero, so audio is a shadow/suggestive cue, not a default-flip.
- `[analysis]` `python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
- `[analysis]` `python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor eval_baselines\heuristic_lovo_n15.json --version 0.1.1-n15-shadow.20260704 --out artifacts\rally_tal_gen0` produced learned LOVO mean F1 `0.551`, floor passed versus heuristic `0.446`, but sign test `9/15`, `p = 0.3036`, `ship=False`.
- `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15>

--features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.
- `[analysis]` Before A6, `python -m backend.eval.ablation --features-dir output --granularity group` failed at import due to a circular import between `backend/eval/ablation.py` and `backend/eval/ablation_pdf.py`; A6 fixes the module entrypoint and keeps full-fusion ablation scoped to the 6 videos with full schema until A7.

4. Launch strategy

4.1 Alpha lane: make the product usable

The alpha lane is successful when an invited tester can open the app, authenticate, upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane should avoid model-quality promises beyond the measured state and should default to the most reliable serving path.

4.2 Quality lane: make the 15-video corpus gate-ready

The quality lane is successful when the 15-video corpus is portable, reproducible, feature-complete, and backed by refreshed baselines. Only then should R2 tolF1, Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.

4.3 Flywheel lane: make tester feedback useful

The flywheel lane is successful when user corrections can flow into the collector/vault path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning, but it should not block the first controlled alpha if manual promotion can bridge the gap.

5. Risks and guardrails

Risk	Why it matters	Guardrail
Launch activity implies model launch approval	The engine docs explicitly say rally-detection product launch is NO-GO until ship target and WASB C3 clear	Keep all alpha copy scoped to invite-only processing, not owned-model claims
Stale local manifests create false confidence	Manifest-driven tools skipped or failed until paths were rebased	Make the corpus manifest portable and add a path/fixture smoke check
15-video corpus is only partially feature-complete	Fusion/person/audio eval is still n=6 today	Regenerate or upconvert full-fusion features for all 15 before fusion decisions
Baseline refresh accidentally rubber-stamps a regression	Nightly runner correctly reported stale baselines	Refresh baselines only after reviewer sign-off and record the command/output
P10/P11 expands scope before hosted alpha is usable	Corpus flywheel is valuable but can consume the alpha launch window	Keep P10/P11 behind explicit owner reopen unless alpha lane is already stable
WASB checkpoint provenance remains unresolved	Blocks commercial sharing of a trained model regardless of F1	Treat C3 as a hard model-release gate; do not hide it inside product launch tasks

6. Honest limits

Completing this doc's alpha lane does **not** prove the owned model is ready, that WASB weights are commercially clean, that the product can serve unbounded public traffic, that multi-sport is ready, or that every tester correction automatically improves the model.

Completing the quality lane does **not** by itself deploy a service. It only makes the measurement surface honest enough for a launch decision.

7. Deliverables & progress tracker

Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the row explicitly names a cross-repo batch.

ID	Deliverable	Repo / owner	Depends on	Gated?	Status	PR
A0	Create this alpha readiness tracker and index it in docs	`badminton-highlight-indexer`	-	No	DONE	#451
A1	Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified	`khelsutra` + `badminton-highlight-indexer`	A0	Owner spend / CF config	DONE	#462/#464/#465/#467/#468/#469/#470/#474/#476; e2e runlog `docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md`
A2	Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing	`badminton-highlight-indexer`	A0	No	DONE	#452
A3	Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus	`sports-data-collector` + vault	A2	Owner upload/storage scope	DONE	collector #62
A4	Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed	`badminton-highlight-indexer`	A2	Reviewer sign-off	DONE	#453
A5	Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for future Gen-0 comparisons	`badminton-highlight-indexer`	A2	No	DONE	#454
A6	Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus	`badminton-highlight-indexer`	A2	No	DONE	#455
A7	Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15	`badminton-highlight-indexer`	A2	GPU/data availability	DONE	#458
A8	Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips	`badminton-highlight-indexer`	A6, A7	Reviewer sign-off	DONE	#458
A9	Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict	`badminton-highlight-indexer`	A5, A8	Owner approval for M0.4 scope	DONE	#458
A10	Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold, significance rule, and what claims are allowed	Owner + `badminton-highlight-indexer`	A4, A5, A8	Owner decision	GATED	-
A11	Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred, document "no commercial model sharing" copy guardrail	Owner/legal	A10	Legal/provenance	GATED	-
A12	Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion, `train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING` M11	`khelsutra` + `badminton-highlight-indexer` + collector	A1, A3	Owner D13 reopen (`khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`)	GATED	-
A13	Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage, known limits, rollback/disable steps, and owner GO/NO-GO	Cross-repo	A1, A4, A10, A11	Owner launch decision	TODO	-

8. Concrete next PR sequence

1. **Docs PR:** land this tracker.
2. **Corpus portability PR:** rebase/portable manifest tooling plus missing-file smoke test.
3. **Hosted alpha PR/batch:** health/auth/upload/process/job/compile/download verification and runbook updates.
4. **Baseline PR:** intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
```

```
count 15 --write output/human_lovo_manifest.local.json
```

The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.

A4 nightly baseline refresh record:

```powershell
python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json --features-dir output --update-baseline
```

Result:

```text
INFO baseline snapshot written to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\eval_baselines\nightly_baseline.json
INFO   default: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
INFO   milestone: n=15 tolF1=0.364 f1@0.5=0.472 merge_rate=0.135
INFO   baseline_off: n=15 tolF1=0.138 f1@0.5=0.237 merge_rate=0.638
```

Pass-mode proof:

```powershell
python scripts\nightly_regression.py --manifest output\human_lovo_manifest.json --features-dir output --no-report
```

Result: `PASS`, exit code `0`, baseline snapshot `2026-07-03`, default/milestone `tol_f1 = 0.3636`, `f1@0.5 = 0.4723`, `merge_rate = 0.1350`; baseline-off `tol_f1 = 0.1382`, `f1@0.5 = 0.2366`, `merge_rate = 0.6380`.

A5 heuristic floor refresh record:

```powershell
python -m backend.eval.calibrate_local --manifest output\human_lovo_manifest.json --metric f1 --floor-out eval_baselines\heuristic_lovo_n15.json
```

Result:

```text
baseline (untuned) mean held-out f1: 0.237
calibrated mean held-out f1:        0.446 +/- 0.239  (gap +0.000)
```

The committed floor is `eval_baselines/heuristic_lovo_n15.json`: `n = 15`, `mean_f1 = 0.4456`, `std = 0.2390`, `metric = f1`, `iou = 0.5`. Every leave-one-video-out fold selected `(0.05, 1.0, 2.0, 0)`.

A6 ablation/R2 record:

```powershell
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Result: the ablation CLI now runs, but full-fusion feature ablation still reports `n=6` because only the original six feature JSONs carry the full fusion schema; A7 must regenerate/upconvert the newer nine before fusion/person/audio promotion decisions. The n=15

R2/tolF1 contrast is positive for the served milestone versus baseline-off: baseline/off `tolF1 = 0.138`, served `tolF1 = 0.364`, `Delta tolF1 = +0.225` with CI `[+0.165, +0.283]`, sign `14/0`, `p = 0.000`; `Delta F1@0.5 = +0.236` with CI `[+0.161, +0.313]`; merge rate improves from `0.638` to `0.135`. Net over-segmentation guard passes (`1.276 -> 0.378`), while the strict segment-count rule remains blocked by increased splits, so R2/tolF1 is the supported gate for the baseline-off to served milestone contrast rather than a blanket fusion/default-flip approval.

A7 full-fusion/audio feature refresh record:

```powershell
python -m backend.eval.fusion_golden --manifest output\human_lovo_manifest.json --out-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --missing-fusion-only --smallest-first --max-gpu-temp-c 87
python -m backend.eval.fusion_audio --manifest output\human_lovo_manifest.json --features-dir output --source-manifest ..\sports-data-collector\data\golden_source_videos.json --augment
```

Result: the GPU pass completed without a thermal abort after fans were set to max performance. Full-fusion schema is now present for `15/15` feature JSONs, and audio features are present for `15/15`. The large regenerated clips completed as follows: `mahadevpura_1` `1` window / `0` positives / `10` GT, `GX020094` `99` / `30` / `40`, `GX030094` `79` / `21` / `41`, `GX010141` `26` / `11` / `29`, `GX010137` `56` / `31` / `34`, `Boxhill_Doubles` `59` / `12` / `43`, `mahadevpura_2` `14` / `2` / `40`, `GX010128` `51` / `15` / `52`, `Badminton_BXH_2` `29` / `1` / `44`.

A3 source-manifest follow-up: `sports-data-collector` PR #62 is merged. The authoritative collector manifest now records `13/15` present local source/proxy paths and two Boxhill run-log-only eval inputs: `GX010141_1080p.mp4` (`daaf8af198640516d14b217f50445d99`) and `GX010137_1080p.mp4` (`74eedbc35d739c231344b8929df4b4a7`). The F-drive 4K originals are preserved there only as `original_*` provenance, not as eval join targets, because the golden trajectories/labels are 1920-wide. Freshly re-extracting those two Boxhill full-fusion files requires recovering the exact 1080p eval inputs or adding a coordinate-normalization fix; do not substitute the 4K originals for person-cue extraction.

A8 fusion/default-decision record:

```powershell
python -m backend.eval.fusion_compare --features-dir output
python -m backend.eval.fusion_audio --features-dir output --compare
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.serve_contrast --manifest output\human_lovo_manifest.json
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Result:

- Person fusion is **not promoted**: shuttle-only F1 `0.302`, shuttle+person F1 `0.311`, `Delta F1 +0.008`, paired CI `[-0.040, +0.055]`, sign `8W/6L`, `p = 0.791`.
- Audio is **shadow/suggestive, not promoted**: shuttle+person F1 `0.311`, shuttle+person+audio F1 `0.329`, `Delta F1 +0.018`, paired CI `[-0.017, +0.053]`, sign `10W/4L`, `p = 0.180`.
- Group ablation remains non-significant for cue promotion: audio `+0.018` CI `[-0.017, +0.053]`; shuttle `+0.013` CI `[-0.002, +0.029]`; gate_a `-0.001` CI `[-0.012, +0.008]`; person `-0.009` CI `[-0.045, +0.021]`.
- Gate A served default is **rejected**: proposal mean `Delta F1 +0.0287`, CI `[-0.0222, +0.0680]`; served mean `Delta F1 -0.0328`, CI `[-0.0799, +0.0052]`, `regresses=True`.
- R2/cap/inactivity remains the strong quality result: served candidate F1 `0.472` vs baseline-off `0.237`, `Delta F1 +0.236`, CI `[+0.161, +0.313]`; tolF1 `0.364` vs `0.138`, `Delta tolF1 +0.225`, CI `[+0.165, +0.283]`; merge rate `0.135` vs `0.638`. Net over-seg guard passes, strict per-video split-count guard still blocks blanket promotion language.

A9 Gen-0 shadow-training record:

```powershell
python -m training.gen0.harness --manifest output\human_lovo_manifest.json --floor eval_baselines\heuristic_lovo_n15.json --out artifacts\rally_tal_gen0 --version 0.1.1-n15-shadow.20260704
```

Result: current Gen-0 LogReg/TAL harness produced local bundle `artifacts\rally_tal_gen0\0.1.1-n15-shadow.20260704` with learned LOVO mean F1 `0.551` versus heuristic floor `0.4456`, so `floor_passed=True`. The gate remains **non-serving**: paired sign test `9/15`, `p = 0.3036`, `ship=False`; blockers are uncalibrated ship-gate target #172 and WASB checkpoint provenance C3. TemporalMaxer itself is not implemented in this branch; the evidence supports either closing A9 as a Gen-0 shadow baseline or opening a follow-up M0.4 scorer-swap PR.

9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should P10/P11 be reopened before any tester wave?
2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU, GCE L4 VM, or another approved GPU host?
3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains unresolved?
4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a checked-in redacted manifest, or both?
5. **Engineering:** Should A9 close on the current Gen-0 non-serving bundle, or should the actual TemporalMaxer scorer swap be implemented before marking A9 done?

10. Definition of done

- [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without SSH.
- [ ] Alpha service is gated behind the chosen auth posture and has a documented disable/rollback path.
- [x] n=15 corpus is portable enough that manifest-driven quality commands do not rely on stale local absolute paths.
- [x] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
- [x] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
- [x] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected on the 15-video corpus.
- [ ] Ship-gate target and allowed alpha claims are recorded.
- [ ] WASB C3 status is recorded and reflected in alpha messaging.
- [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for alpha-plus.
- [ ] Owner records GO/NO-GO for the first alpha wave.

11. References

Internal docs:

- `docs/NEXT_STEPS.md`
- `docs/ROADMAP.md`
- `docs/data_pipeline/GOLDEN_VIDEOS.md`
- `docs/R2_EVAL_METRIC_DESIGN.md`
- `docs/RALLY_DETECTION_FIXES_PLAN.md`
- `docs/COMPUTE_DECOUPLED_SERVING/README.md`
- sibling `khelsutra/NEXT_STEPS.md`
- sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`
- sibling `khelsutra/docs/PER_RALLY_SELECTION.md`
- sibling `sports-data-collector/data/golden_source_videos.json`

Local verification commands used for the 2026-07-03 snapshot:

```powershell
```

```
python -m backend.eval.fusion_compare --features-dir output
python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor eval_baselines\heuristic_lovo_n15.json --version analysis-n15 --out <temp-out>
python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir output --no-report
python -m backend.eval.ablation --features-dir output --granularity group
python -m backend.eval.rally_seg_eval --manifest output\human_lovo_manifest.json --features-dir output --preset served --max-window-duration 0 --inactivity-end 0 --vs served --vs-max-window-duration 6 --vs-inactivity-end 1.5 --json-out output\a6_r2_ablation.json
```

Changelog

- 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`, golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
- 2026-07-04 - Recorded A7 full-fusion/audio regeneration, A8 n15 fusion/default decisions, and A9 Gen-0 n15 shadow-training gate verdict.
- 2026-07-04 - Marked A3/A7/A8/A9 complete after collector #62 and indexer #458 review/CI; clarified that the two Boxhill eval inputs are run-log-only 1080p hashes and that F-drive 4K originals are provenance-only.

5. user (2026-07-04T11:48:46.562Z)

Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)

The cloud-serving compute path: a control plane (`CloudRunCompute`, `backend/compute/cloud_run.py`)
dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then **bucket-polls**
a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.

> **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real build + push +
> run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ? confirm cmd + hourly
> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not** build this image.

0. Prerequisites (owner)
- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs enabled.
- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore ? see
  `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated): `gs://<bucket>/models/wasb_badminton_best.pth.tar`.

1. Build + push the image (Artifact Registry)
```bash
PROJECT=<gcp-project>; REPO=rally
# L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
REGION=asia-southeast1
gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION 2>/dev/null || true
# `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
```
(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per D17.)

2. Create the Cloud Run **Job** (GPU, scale-to-zero)

The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region bucket** mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step needed).
```bash
gcloud run jobs create rally-worker \
  --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
  --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
  --cpu 8 --memory 32Gi \
  --max-retries 0 --task-timeout 3600 \
  --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
  --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
  --add-volume-mount volume=serving,mount-path=/gcs
```
- **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it also halves the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive Y/n which, in a non-interactive shell, **hangs forever with zero output and nothing created**. An empty `gcloud` that "did nothing" ? suspect a hidden prompt.
- 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU headroom is what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives in memory, so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).

3. Wire the dispatcher (control plane)
The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-serving` ? `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute` from env:
```bash
export DEPLOYMENT_PROFILE=cloud-serving
export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION GOOGLE_CLOUD_PROJECT=$PROJECT
# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job instead of
# running in-process; the container runs the SAME worker INLINE (compute_target forced local_gpu).
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri gs://<bucket> --segmenter wasb_hybrid
```
The AI handoff needs its key IN THE JOB env (`GEMINI_API_KEY` ? mount a Secret Manager secret on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate windows emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to a silent 0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail this at boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.

3a. Control-plane SERVICE deploy ? required flags for reliable async jobs

The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-PROCESS**: the request returns `202` and a background thread drains the job (GPU detection dispatches to the Job; the reel stitch runs on the control plane itself). On Cloud Run that background thread only makes progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:

The canonical deploy path is the committed manifest generator (#473) ? it bakes the

cloud-serving config.json into a startup wrapper, sets every flag below, pins `maxScale=1` (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB` (a reserved runtime env name that gets a manifest rejected; the code default is already right):
```bash
python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
gcloud run services replace service.yaml --region $REGION   # from PowerShell on Windows
```
`--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with `on` before testers touch the service again. The flag-based equivalent (for reference ? a manifest round-trips the bash wrapper's metacharacters, flags don't on Windows):
```bash
gcloud run deploy rally-control-plane --image <image> --region $REGION \
  --no-cpu-throttling \        # CPU stays allocated between requests ? the drain thread finishes
  --min-instances 1 \          # one always-warm instance drains reliably (cheap CPU; NOT the D7 GPU pool)
  --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
  --set-env-vars DEPLOYMENT_PROFILE=cloud-serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
```
Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the reel never finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT contradict D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-in-depth is in the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a restart (idempotent stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on the next drain.
Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to the CF proxy (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so flipping `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify works out of the box.

4. M7 acceptance (the owner runlog)
Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact commands, rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test (`ffmpeg -f null -` exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.

Gotchas
- The dispatched container **must run inline** ? `CloudRunCompute` forces `compute_target=local_gpu` + `indexing.detector_impl=native` in the per-execution args so the Job never re-dispatches itself.
- Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud Run API status ? so it reuses the storage layer + `execution_policy.poll_until` and survives a control-plane restart.
- A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker** carrying the real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min hang on a bad video). Only a **hung/crashed** container (no marker at all) makes the bounded poll

```
time out
      (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and the API path
      surfaces it as a `cloud dispatch error` job failure (not a traceback).
    - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default libx264.
    - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ? `GoogleCloudRunJobClient.run_job`
      fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real `job_path`;
      an unset project would otherwise build a bogus `projects/None/...` name). Export it (step 3) before
      dispatching. The guard runs before the `google-cloud-run` import, so it is exercised in CI even
      though the SDK isn't a CI/test dependency.
    - **Hand-rolled `jobs execute --args` with `gs://` URIs:** gcloud's default comma-split plus its
      alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser, and a
      `^:^` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in the args,
      e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have this
      problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must include the
      dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
      `indexing.detector_impl=native`.
    - **Git Bash (MSYS) on Windows mangles Unix-looking values** before gcloud sees them:
      `--add-volume-mount ?mount-path=/gcs` became `C:/Program Files/Git/gcs`, and
      `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then exec-fails).
      Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks gcloud's own
      wrapper). PowerShell has its own trap: quote any flag value containing commas.
    - **First execution after an image push** spends minutes in "Container image import in progress"
      (visible in `executions describe status.conditions`, NOT in logs) ? that's progress, not a hang;
      later executions reuse the imported image.

```

6. user (2026-07-04T11:48:52.503Z)

```
    # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7, ADR-012/D16).
    #
    # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-located so
    # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the owner on the
    # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too heavy) ? the
    # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
    #
    # Two python stacks, deliberately separate (a framework conflict otherwise ? same split as the
    # native GPU box, deploy/digitalocean/gpu_setup.sh):
    #   * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
    #   * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as a subprocess
    #     (the native runner shells out to $WASB_PYTHON), so the app's torch never co-installs with it.
    #
    # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT cleared for
```

```
#   commercial sharing. They are NOT baked into this image; stage them at runtime (a bucket fetch or a
#   mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner serving.

FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

ENV DEBIAN_FRONTEND=noninteractive \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    CONDA_DIR=/opt/conda \
    WASB_REPO_DIR=/opt/models/WASB-SBDT

# ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB env build.
RUN apt-get update && apt-get install -y --no-install-recommends \
        ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
        python3 python3-pip python3-venv \
    && rm -rf /var/lib/apt/lists/*

# --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a, ADR-005) ---
# Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ? stays
# MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's Anaconda-ToS-gated
# `defaults` channels (the ?200-employee Terms-of-Service trap flagged in PACKAGING_PLAN.md §3).
# MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so $WASB_PYTHON below
# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are unaffected by the switch.
ENV MAMBA_ROOT_PREFIX=/opt/conda
RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
        | tar -xj -C /usr/local bin/micromamba \
    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
    && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url https://download.pytorch.org/whl/cu113 \
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib "numpy<1.24"

# --- Main application env ---
WORKDIR /app
COPY pyproject.toml README.md ./
COPY requirements-trackers.txt ./
COPY backend ./backend
COPY training ./training
# Install with the [storage-gcs,cloud-run,auth] extras. storage-gcs: the cloud worker PULLS the gs://
# input and PUSHES the gs:// outputs/done-marker through backend.storage (google-cloud-storage is
# lazy-imported there); without it the job dies at the first gs:// fetch with `ModuleNotFoundError:
# google.cloud`. cloud-run: this SAME image is reused as the deployed CONTROL PLANE, which dispatches
# the L4 worker Job via google.cloud.run_v2 (backend/compute/cloud_run.py, lazy-imported); without it
# the control plane fails every dispatch with "google-cloud-run is required for CloudRunCompute's real
# client". Both are real Cloud Run failures the mock-client tests can't see (the SDKs
```

```
are faked in CI).
# auth: PyJWT[crypto] for M9 edge-auth (security.edge_auth_enabled ? the Cf-Access JWT verify the
# reused-as-control-plane deployment runs behind api.khelsutra.guru); jwt is lazy-imported and
# edge-auth is default-OFF, so the worker path is unaffected ? but without it a control-plane
# deploy with edge-auth ON needed a hand-built derived image (the A1 deploy's PyJWT overlay).
# D13/P12: `trackers` (ByteTrack's new home) is a git-only dep (broken PyPI wheel ?
# roboflow/trackers#474) so it's NOT pulled by `pip install .`; install it out-of-band before
# the main env, or the person-cue path crashes with ModuleNotFoundError. The immutable git SHA is
# single-sourced in requirements-trackers.txt.
RUN python3 -m pip install --upgrade pip \
    && python3 -m pip install -r requirements-trackers.txt \
    && python3 -m pip install ".[storage-gcs,cloud-run,auth]"

# The native runner finds the WASB env + repo by env (no `conda activate` needed); weights come in at
# runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the default.
# PYTHONUNBUFFERED: without it the worker's stdout/stderr sit in the block buffer until process
# exit, so Cloud Logging shows a single end-of-run burst instead of live progress (observed on the
# first real M7 runs, 2026-07-03) ? a running job looks stalled and mid-run failures log nothing.
ENV PYTHONUNBUFFERED=1 \
    WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
    WASB_REPO_DIR=/opt/models/WASB-SBDT \
    WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
    RALLY_DETECTOR_IMPL=native

# The dispatcher (CloudRunCompute) supplies the per-execution args:
#   infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid --set ?
ENTRYPOINT ["python3", "-m", "backend.worker"]
```

7. user (2026-07-04T11:48:54.792Z)

472 OPEN ops: api.khelsutra.guru Google-managed cert auto-renew (~90d) can stall behind the proxied CF CNAME ? verify before ~2026-09-30 2026-07-04T08:52:38Z
471 OPEN cloud-serving: control-plane state (SQLite jobs/videos DB) is per-instance ephemeral ? maxScale>1 splits the queue and /api/jobs polls bug 2026-07-04T08:52:15Z
448 OPEN Detach post-upload processing into a durable background job UX bug 2026-07-03T10:34:06Z
447 OPEN Add API support to delete a video from the Studio library bug 2026-07-03T08:25:17Z
445 OPEN Make eval/litmus consumers fetch gs:// labels/trajectories/videos by URI (not local open()) 2026-07-02T16:14:39Z
444 OPEN Corpus Phase-2: decide + execute the large source-video upload to GCS (owner-gated) 2026-07-02T16:14:37Z
443 OPEN [security] Default CORS allow_origins=['*'] lets any site script the localhost appliance (R1-12) 2026-07-02T16:12:07Z
384 OPEN ci: single-source the shared offline guard lanes + coverage floor via a workflow_call reusable 2026-06-25T17:23:52Z
349 OPEN WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode) 2026-06-23T16:24:58Z
332 OPEN [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending it to Gemini 2026-06-22T14:18:42Z
331 OPEN [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)

2026-06-22T14:14:15Z
328 OPEN Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs 2026-06-22T13:57:58Z
327 OPEN Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage backends from the path-jail 2026-06-22T13:54:42Z
326 OPEN Hosted-app observability: request tracing + replay for debugging production CUJ failures 2026-06-22T13:54:40Z
301 OPEN T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha) enhancement 2026-07-02T15:00:45Z
175 OPEN Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev sessions 2026-07-02T15:00:46Z
172 OPEN Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified) 2026-06-14T16:40:59Z

8. user (2026-07-04T11:49:07.085Z)

===== ISSUE #471 =====
title: cloud-serving: control-plane state (SQLite jobs/videos DB) is per-instance ephemeral ? maxScale>1 splits the queue and /api/jobs polls
state: OPEN
author: avidullu (Avi)
labels: bug
comments: 0
assignees:
projects:
milestone:
number: 471
--

Symptom

The A1 control-plane service (rev 00004) runs with `min-instances 1` but **maxScale 3**. All server state ? the jobs queue, the videos/segments tables (SQLite), and scratch files ? lives in the instance''s ephemeral `/tmp`. With more than one instance:

- `/api/process` returns 202 from instance A; the client''s `/api/jobs/{id}` poll can land on instance B, which has never heard of the job ? 404 / empty status while the job is actually running.
- A restart or scale-down loses the queue outright (partially mitigated: `recover_stale_jobs()` re-queues interrupted compiles, but only on the instance that restarts, and completed-job history is gone).

#463 (reel mirror to gs://) and #461 (upload staging to gs://) fix the **artifacts** outliving an instance, but not job/queue **state** visibility across instances.

Options

1. **Alpha-cheap:** pin `maxScale=1` in the service manifest + document the single-instance constraint (matches the current 4-email alpha traffic; `min-instances 1` already implies a warm single box).
2. **Durable:** move job/video state to a shared store (Cloud SQL / Firestore / a GCS-backed job store). Pairs with #448 (durable background-job UX) and #326 (request tracing).

Until one lands, treat any multi-instance scale-out of the control plane as broken for the async-job CUJ.

Found during the A1 alpha deploy (runlog #464).
===== ISSUE #472 =====
title: ops: api.khelsutra.guru Google-managed cert auto-renew (~90d) can stall behind the proxied CF CNAME ? verify before ~2026-09-30
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:

projects:
milestone:
number: 472
--
Watch-item from the A1 deploy (2026-07-04)

`api.khelsutra.guru` fronts the Cloud Run control plane via a **domain mapping** (CNAME `ghs.googlehosted.com`, **PROXIED** through Cloudflare) + a Google-managed certificate. Initial issuance succeeded, but Google-managed certs renew ~every 90 days and the renewal validation can fail when the CNAME is proxied (Google''s challenge sees Cloudflare''s edge, not the expected record).

Action

- **Before ~2026-09-30:** check the domain-mapping cert status (`gcloud beta run domain-mappings describe --domain api.khelsutra.guru --region asia-southeast1`) and confirm renewal isn''t stalled.
- If it stalls: either temporarily grey-cloud (DNS-only) the CNAME so the renewal validation passes, or switch to a **Cloudflare Origin CA cert** on the mapping so renewal never depends on what Google can see through the proxy.

Context: session notes from the A1 alpha deploy (runlog #464). CF Access (M9) is unaffected either way ? this is only about the Google-side TLS cert on the mapping.
===== ISSUE #443 =====
title: [security] Default CORS allow_origins=['*'] lets any site script the localhost appliance (R1-12)
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 443
--
Audit finding R1-12 (`docs/AUDIT_REPORT_khelsutra-guru_2026-07-02.md`), the one confirmed-but-untracked security residual after the 2026-07-02 remediation.

Issue
`backend/main.py` builds the CORS middleware with `allow_origins=_cors_origins_from_env()`, whose default is `["*"]`. On the local appliance (`indexer --app`) this means **any website the user visits can script the loopback API** ? trigger paid Gemini runs, read local video/segment data ? via cross-origin requests.

Mitigation already in place / why P2 not P1
- `allow_credentials=True` + `*` is rejected by browsers, and Chromium's Private Network Access blocks public?localhost preflights in Chrome/Edge (dominant share). Residual exposure is mainly older Firefox / non-PNA browsers.
- The bundled SPA is same-origin, so it does not need the wildcard.

Proposed fix
- Default to same-origin/loopback (e.g. `[]` + skip the middleware, or an explicit `http://localhost:*` dev list); keep `RALLY_CORS_ALLOW_ORIGINS` as the widening knob for hosted deploys.
- Emit a startup exposure warning when the wildcard is active on a non-hosted profile.
- Regression test: an evil-origin preflight is rejected under the default.

Deferred from the audit remediation because changing the default has appliance-UX implications worth a deliberate call.
===== ISSUE #332 =====
title: [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending it to Gemini
state: OPEN

author: avidullu (Avi)
labels:
comments:	0
assignees:
projects:
milestone:
number: 332
--
Ask (owner)
A **litmus test** that flags if a video is **not a racket-sports game**, so we **don't send inappropriate content (NSFW/porn) to Gemini** ? *"some users will try."* Acknowledged as non-trivial; design later.

Why it must run BEFORE Gemini
1. **ToS/abuse** ? sending explicit content to Google's API is a real violation/abuse risk.
2. **Cost** ? Gemini is paid; garbage in = wasted spend.
3. **Quality** ? non-sports input yields nonsense rallies.

Proposed design (first pass ? likely graduates to a tracked design doc)
Integration point: a new **content-safety validator** in the *existing validation phase* (the pipeline already does `Running validations?` ? `passed checks` ? segment). Reject before the gemini segmenter with a clear, friendly message (loud-failure bar). Cheap + **fully local** (no Gemini call for rejected content), on a handful of sampled frames.

Two gates:
1. **NSFW / abuse gate (the porn concern ? must-have):** a fast image classifier on ~N sampled frames ? reject if explicit. *Licensing constraint:* weights must be **MIT/Apache/BSD** (the commercial-clean guardrail) ? the model choice needs license vetting.
2. **Sports-positive gate (quality/cost filter):** "does this look like a racket-sports match?" ? **reuse the engine's existing court detection + person detection + motion**; if no court / no players / no rally-motion consistently across sampled frames ? flag as not-a-match.

Tuning / trade-offs: sample count + thresholds tunable; bias the **NSFW gate toward catching** (abuse cost is high ? a few false rejections are acceptable), bias the sports gate toward **allowing** (don't reject a real-but-unusual match). Provide a clear rejection reason in the result + (per #328) the trace.

Licensing/guardrail: no paid-LLM-as-classifier for training; any bundled weights MIT/Apache/BSD; exclude minors from any training pool (existing guardrails apply).

Relates to: the validators layer (the integration seam), #328 (surface the rejection reason/trace).

9. user (2026-07-04T11:49:12.325Z)

===== ISSUE #327 =====
title: Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage backends from the path-jail
state: OPEN
author: avidullu (Avi)
labels:
comments:	0
assignees:
projects:
milestone:
number: 327
--
Today
When `security.allowed_video_root` is set (required to expose), `/api/process` runs `validate_input_path` (local-under-jail) **before** resolving the storage scheme, so `gs://bucket/x.mp4` ? `os.path.realpath` escapes the jail ? **400**. Bucket ingest is therefore impossible on the exposed alpha box. Pinned by `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri` (gs/s3/gdrive all

400).

Want

The owner wants to ingest large videos via `gs://` on the hosted box (far more convenient than a 500 MB browser upload).

Proposed

Make `validate_input_path` / the `/api/process` preamble **scheme-aware**: either (a) allow recognized storage schemes (`gs://`, `s3://`, `gdrive-api://`) *before* the local-jail check, or (b) apply the jail **only to local refs** (validate after `storage.resolve_input_ref`, jailing only `backend == "local"`).

? Must NOT reopen the arbitrary-local-file hole ? the jail must still fully apply to **local** paths (traversal/symlink/absolute). Add tests for both the allow (bucket URI accepted) and the reject (local traversal still 400) paths. Pairs with the `gs://` ingest the alpha CUJ + DEPLOY_ALPHA §7 currently disclaim.
===== ISSUE #301 =====
title: T11 follow-up: run the upload retention sweep as an automated scheduled job (hosted alpha)
state: OPEN
author: avidullu (Avi)
labels: enhancement
comments: 1
assignees:
projects:
milestone:
number: 301
--

Context

The T11 **upload retention sweep** landed in #299 (merged, `cb7394b`): `tools/cleanup_caches.py` was extended to purge ? past a retention window ? uploaded source videos and orphaned `.partial-<id>` files in `security.upload_dir` (default `output/uploads`), alongside the existing regenerable-cache cleanup. It is **dry-run by default**, keeps reels/DB/CSVs durable, and emits a machine-readable plan via `--json`.

That PR delivered the **tool**. What it did **not** do is run it **unattended**: today an operator must manually run `python -m tools.cleanup_caches --apply` on the box. Before/while non-owner (alpha) users upload footage, the retention window should be enforced automatically without a human in the loop.

Goal

Wire the existing sweep into an **automated scheduled job** so stale uploads are purged on the owner-set retention window (uploads **7d**; reels kept durable) without manual intervention, with loud structured logging captured to observability.

Do not re-implement the sweep ? schedule the existing tool. The `--json` output (`{windows, uploads_dir, reclaimable_bytes, purge[], by_category[]}`) is already shaped for a wrapper/log sink.

What's needed

- A scheduler appropriate to the deployment:
  - **Single-box alpha** (engine on one host): a host **cron** or **systemd-timer** entry that runs the sweep daily.
  - **Cloud Run serving path**: a **Cloud Scheduler ? job** invocation (or a sidecar), consistent with `COMPUTE_DECOUPLED_SERVING/`.
- The scheduled invocation runs `--apply` with `--uploads-retention-days 7` and `--uploads-dir` matching `security.upload_dir` (consider resolving from config rather than hardcoding the path).
- **Default-OFF / opt-in** so the single-user local install is never surprised by an unattended deletion job.
- **Observability**: capture the `--json` plan + the per-item `PURGED/FAILED` lines to logs each

run (counts + bytes freed + the active windows), per the launch quality bar (loud, structured, correlation id).
- **Document** it in the deploy runbook (engine `deploy/` and/or `khelsutra/deploy/DEPLOY_ALPHA.md`).

Design considerations / open questions

- **In-flight safety:** the retention window already protects recent `.partial-<id>` files (an in-flight chunked upload < window is never swept). Confirm the cadence + window leave a comfortable margin over the max upload duration.
- **Active-job safety:** a *finished* upload may still be referenced by an in-progress `/api/process` job. Decide whether the scheduled apply should additionally skip uploads referenced by a non-terminal row in the `jobs` table (defense-in-depth beyond the time window). The window alone is probably sufficient for 7d, but worth a deliberate call.
- **Per-user subdirs (PR-3):** the sweep currently leaves an unknown upload **subdir** untouched (`classify` returns `upload subdir` = keep). When identity scoping lands, the job/tool will need to recurse per-user.
- **Apply guard:** since the scheduled job runs `--apply`, gate it behind an explicit config flag + keep the conservative window so a misconfiguration can't nuke fresh uploads.

Acceptance criteria

- [ ] A scheduled mechanism runs `tools/cleanup_caches.py` on a cadence (e.g. daily) in the hosted/alpha deployment.
- [ ] It applies the owner-set window (uploads 7d, reels durable) and writes the structured `--json` plan + outcome to logs/observability.
- [ ] Default-OFF / opt-in; the local single-user path is unaffected.
- [ ] In-flight + active-job safety is reasoned about and documented (window margin, optional jobs-table check).
- [ ] The deploy runbook documents how to enable + verify it.
- [ ] A test/smoke confirms the wrapper invokes the tool with the right flags (the tool's own logic is already unit-tested).

References

- PR #299 (the tool) · `tools/cleanup_caches.py` · `tests/test_cleanup_caches.py`
- `backend/api/uploads.py` (where uploads land) · `backend/config/models.py` `SecurityConfig.upload_dir`
- Cache-hygiene policy (retention table) · `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` §9 (T11, khelsutra repo) · the `ui-driven-cloud-serving-gap` thread

Filed as the scheduling follow-up to #299; the sweep tool itself is done and merged.
===== ISSUE #328 =====
title: Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 328
--
Feedback from the first real hosted-alpha use. After uploading a video, the owner wanted to **see the runlog and trace the execution** for *that* video ? but today the trace only lives in `journalctl -u khelsutra-engine` and the per-video `obsreport` on the box, neither user-accessible.

What the trace contains (visible in the engine log today): validation ? segmenter selected ? video duration + chunking plan ? per-chunk Gemini extraction/analysis ? rallies found ? reel compile. Rich and useful ? just not surfaced.

Proposed:

1. An endpoint to fetch a job's execution trace, e.g. `GET /api/jobs/{job_id}/trace` (or expose the per-video `obsreport` / `report.json`) ? structured steps + timings + counts.
2. A UI affordance ("view processing details" / a live progress log) on the job card so a user can watch/trace their video.
3. Correlate with the `x-correlation-id` so a UI trace ties to the server log.

Relates to #326 (request tracing/replay observability) ? this one is the **per-video pipeline** trace specifically.
===== ISSUE #326 =====
title: Hosted-app observability: request tracing + replay for debugging production CUJ failures
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 326
--
Context
A hosted alpha upload failure was hard to debug because the failing requests **never reached the engine** ? Cloudflare Access `302`'d them at the edge, so `journalctl` showed nothing. We want first-class observability so production CUJ failures are diagnosable (and replayable) without guesswork.

What exists today
- `x-correlation-id` threaded SPA ? engine.
- Structured engine logs (`journalctl -u khelsutra-engine`) ? every request method/path/status.
- Per-video `obsreport` (observability report next to each processed video).
- **Cloudflare Zero Trust ? Logs ? Access** ? the edge allow/block/login log (where the cross-origin failure was actually visible).

Gap
- No unified **request capture/replay** store for the app's API requests.
- No easy way to **import/replay** captured requests for repro or regression.
- Edge (Access) failures aren't correlated with engine logs.

Proposed
1. Optional request-capture middleware ? a debug store (method, path, status, cid, redacted headers, timing), **default-OFF**, flag-gated.
2. A **replay harness** that loads captured requests and re-issues them against a target engine (repro + CUJ regression).
3. A debugging **runbook**: Access logs + `journalctl` + correlation-id flow.
4. Basic live-droplet monitoring (uptime / job-failure / Gemini-cost), ties to the alpha-ops hygiene item.

10. user (2026-07-04T11:49:27.456Z)

===== ISSUE #331 =====
title: [perf] Hosted processing is CPU-bound (ffmpeg/cv2) ? measured per-phase breakdown + levers (GPU/NVENC, parallel extraction, async compile)
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 331
--
Measured on the live alpha (2-core CPU droplet), a real 411s (6.9 min) 1080p match ? 26 rallies

Phase	Wall time	Bound by	Lever
Upload (7 chunks)	~30 s	client upstream bandwidth	(network, not us)
Validation	~87 s	**CPU** (frame decode/validators)	GPU/NVDEC, faster CPU
Chunk extraction (6× ffmpeg re-encode + downscale)	**~5.5 min**	**CPU** (software libx264)	**GPU/NVENC**, more cores, **skip the downscale**, parallelize
Gemini upload + analysis	~1.5 min	network (upload) + Gemini API (~15s/chunk, parallel)	(fixed ? Google's side)
Reel stitch (trim+re-encode 10 clips ? concat ? watermark)	~3 min	**CPU** (per-clip re-encode)	**GPU/NVENC**; also async-compile (#329) so it doesn't 524

~65% of the wait is one thing: ffmpeg re-encoding on 2 cores (extraction + per-clip trim). The Gemini *analysis* is fast and external; the upload is client bandwidth.

Recommended levers (in impact order)

1. **GPU box (L4) with NVENC/NVDEC** ? collapses extraction + stitch from minutes to seconds; **also unlocks the owned WASB detector** (higher rally-detection quality) ? speed **and** quality. (Per ADR-012/013, GCP L4 / Cloud Run GPU.)
2. **Software (no new HW):** skip/relax the `ai_scale_width` chunk downscale (trade upload bytes for far less encode); parallelize extraction across cores; faster encode preset.
3. **Async `/api/compile`** (#329) so a slow stitch returns 202 + poll instead of a Cloudflare 524.

Baseline numbers above are a clean before/after reference once a GPU/optimizations land.
===== ISSUE #349 =====
title: WASB native: decode is CPU-bound & single-threaded in streaming mode ? L4 GPU ~80% idle (no NVDEC/parallel decode)
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 349
--

Summary

WASB native detection on the L4 is **decode-bound on the CPU**, not GPU-bound. On the streaming path the frame decode runs **single-threaded** (`num_workers` is forced to 0), so the L4 sits mostly idle (~20% average, bursting 0?100?0) while one CPU core does HEVC/H.264 decode via `cv2.VideoCapture`. There is **no GPU/NVDEC decode path** anywhere in the engine. Throughput on a 1080p HEVC clip was ~2× realtime (5:52 video ? ~12.5 min wall-clock).

Evidence (live, on the L4 VM)

- `nvidia-smi dmon` during a run: `sm%` bursts `0 ? 85 ? 100 ? 25 ? 0` ? the L4 does a forward pass, then waits on the next decoded batch. Average ? 20%.
- The `wasb_infer.py` process ran at ~198% CPU; GPU temp 47?77 °C, ~4.2 GB resident.
- 5:52 (352 s) 1080p HEVC, 691 MB ? started 13:50, finished 14:02 (~12.5 min).
- Installed OpenCV in the wasb env (`4.13.0`, headless PyPI wheel) reports `hasattr(cv2, "cudacodec") == False` ? no GPU decode even if code asked for it. (System `ffmpeg` *does* list `cuda` as a hwaccel, but the engine never routes decode through it.)

Root cause

1. **All decode is CPU `cv2.VideoCapture`** (OpenCV's bundled ffmpeg). No `cv2.cudacodec` / NVDEC / `-hwaccel cuda` / decord-GPU anywhere in `backend/`.
2. **Streaming forces single-process decode.** `wasb_infer.run()` sets `loader_workers = 0 if streaming else num_workers` (the in-memory frame store + the `cv2.imread` shim can't be pickled to DataLoader workers). So the fast (no-PNG) path is also the *least parallel* one.
3. The disk-frames path (`stream_video=false`) **can** use `num_workers>1` to parallelise decode/read and keep the GPU fed, but pays a full PNG extraction first (slow + disk). The

2026-06-20 deploy report already noted streaming "starves the GPU" on long clips and recommended the disk path for them.

Options (in rough order of effort)

- **Config-only mitigation (testable now, a trade-off, not a fix):** `indexing.wasb.stream_video=false` + `indexing.wasb.num_workers>1` ? parallel CPU decode keeps the L4 fed. Costs a full PNG extraction + disk. (Side benefit: also dodges the streaming resume-cache bug ? see companion issue.)
- **Batched/threaded streaming decode:** prefetch + decode frames on a small thread pool feeding the existing in-memory shim, so streaming isn't single-threaded. Keeps the no-PNG win without the disk cost.
- **True GPU decode (NVDEC):** decode on the L4 via `cv2.cudacodec.VideoReader` (needs a CUDA-built OpenCV), `PyNvVideoCodec`/`decord`-GPU, or an `ffmpeg -hwaccel cuda` pipe. Biggest win on long clips; biggest change (new dep + tensor-on-GPU handoff, must preserve the byte-identical frame contract the pipeline relies on).

Impact / severity

High for cost/throughput on the GPU-VM profile: you pay ~$0.70?0.85/hr for an L4 that's ~80% idle during detection because decode is the bottleneck. Fixing decode parallelism (or NVDEC) is the main lever to cut per-match wall-clock and make the L4 worth its cost.

Environment: GCP L4 VM (`g2-standard-4`), wasb env torch 1.11.0+cu113, OpenCV 4.13.0 (no cudacodec), `stream_video=true`, observed 2026-06-23.

===== ISSUE #175 =====
title: Shared cloud container (OneDrive) for GPU-extractor golden data ? sync GPU box ? dev sessions
state: OPEN
author: avidullu (Avi)
labels:
comments: 1
assignees:
projects:
milestone:
number: 175
--
Problem. The rally-quality measurement is split across machines by capability:
- `backend/eval/fusion_golden.py` (shuttle+person feature **extraction**) needs the videos, trajectory CSVs, `.rallies.csv` golden labels, and a GPU/WSL stack ? **GPU box only**.
- `backend/eval/fusion_compare.py` / `backend/eval/ablation.py` (the LOVO **litmus** ? ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the small `*_golden_features.json` ? **any session** (incl. a CPU dev container).

Today these artifacts live in gitignored `output/` per-machine + scattered local folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs them, so a CPU dev/agent session can't see the GPU box's extraction.

Plan. Use a OneDrive shared folder as the canonical exchange: `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (present + signed-in on both machines under the same account ? auto-syncs). GPU box writes `*_golden_features.json` (+ trajectory CSVs, manifests) there; CPU sessions read from it. Raw videos stay local (tens of GB). Full design + workflow + guardrails: **`docs/GOLDEN_DATA_SHARING.md`**.

Tasks:
- [x] Stand up `OneDrive\rally-annotator\golden-data\{features,trajectories,manifests,baselines}\` + a README (this session).
- [ ] GPU box: run `fusion_golden --out-dir <shared>/features/<tag>` (or extract local then copy the JSONs); confirm it syncs to the dev machine.
- [ ] CPU session: `fusion_compare --features-dir <shared>/features/<tag> --record` + `ablation --granularity group` ? real person-feature ?F1; light up the `skipif` golden-litmus tests.
- [ ] (optional ergonomics) a `RALLY_GOLDEN_DIR` env var honored by `fusion_golden`/`fusion_compare`/`ablation`; a tiny `scripts/sync_golden.py` copier.

- [ ] Keep the folder **private** (Proprietary corpus) and **outside the repo** (never committed).

Refs: `docs/GOLDEN_DATA_SHARING.md`, the NEXT_STEPS "Next-session pickup" anchor, the P3 fusion litmus.
===== ISSUE #172 =====
title: Calibrate the ship-gate F1@tIoU target (currently provisional ? NOT owner-ratified)
state: OPEN
author: avidullu (Avi)
labels:
comments: 0
assignees:
projects:
milestone:
number: 172
--
Context. `training/gen0/harness.py::gate_verdict` references ship-gate F1@tIoU figures sourced from the merged plan (`docs/POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md`, M0):

- Floor (necessary): heuristic LOVO ~**0.480**
- Multi-court reference ("worth serving"): **0.66**
- Proposed for high-P/R + significance: **F1@tIoU0.5 ? 0.70** on multi-court folds, paired-sign p<0.05 vs heuristic

These numbers are PROVISIONAL and have NOT been ratified by the owner. They were listed in the plan as references/proposals derived from a tiny corpus; the owner has explicitly stated they did not commit to that figure. PR #170 should not be read as owner ratification of the gate target.

Ask. Calibrate and ratify the ship-gate target properly once the golden corpus grows to a defensible multi-court size:
- Pick the floor and the "worth serving" threshold from real multi-court LOVO data (not the n=2 synthetic / tiny-corpus numbers).
- Decide the significance bar (paired sign test / bootstrap CI) at that corpus size.
- Until then, `gate_verdict` should treat the F1 target as **uncalibrated/provisional** (a blocker that keeps `ship=False` on F1 grounds), not as "ratified".

Refs: PR #170 (clears the old hard `UNDEFINED` block), plan M0 quality table, #169 (merged plan). C3 (WASB provenance) remains the separate commercial blocker.
===== ISSUE #447 =====
title: Add API support to delete a video from the Studio library
state: OPEN
author: avidullu (Avi)
labels: bug
comments: 1
assignees:
projects:
milestone:
number: 447
--
Bug

KhelSutra Studio has a `Your videos` gallery backed by `GET /api/videos`, but there is no supported API path to delete a stale/failed video from the local library.

During local CUJ testing, a video whose processing job failed before Gemini was configured remained visible in the Studio gallery after a clean restart. The only way to clear it was to manually delete rows from `output/sports_indexer.db` (`jobs`, `videos`, and cascading segment tables), which is not acceptable for operator workflows.

Repro

1. Start the engine and Studio.
2. Upload/process a video.

3. Let processing fail or produce an unwanted stale entry.
4. Restart the engine/Studio.
5. Open Studio root and inspect `Your videos`.

Actual

The old video remains in `GET /api/videos`, and Studio has no delete action/API to remove it.
Operators are forced to re-use a stale record or manually mutate SQLite.

Expected

Provide a supported delete path, for example:

- `DELETE /api/videos/{video_id}` deletes the DB video row and cascades dependent `play_segments`, `candidate_segments`, and related job metadata.
- Optional request/flag to also delete the stored source media and generated reels for that video.
- Studio should expose a delete/remove action in `Your videos` with confirmation.

Notes

The current schema already has foreign keys from `play_segments` and `candidate_segments` to `videos(id) ON DELETE CASCADE`, so the backend may only need a safe endpoint plus cleanup decisions for `jobs` and stored files.

11. user (2026-07-04T11:49:27.599Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:3:Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:57: max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:58: part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:122: "max_part_mb": part_bytes // (1024 * 1024),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:153: f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:257: # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:364: # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:370: max_upload_mb: int = 4096
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:371: max_part_mb: int = 95
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:23:MAX_UPLOAD_MB = 500
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:126:            size_mb = os.path.getsize(video_path) / (1024 * 1024)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:128:            size_mb = 0.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:129:        if size_mb > MAX_UPLOAD_MB:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:131:                f"[{log_prefix}] {video_path} is {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap ? "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\providers\gemini.py:135:            metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:68:        need_mb = need / (1024 * 1024)
```

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py:69:        free_mb = free / (1024 * 1024)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py:74:            need_mb,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py:75:            free_mb,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py:80:                f"Not enough free disk space for {stage}:
need ~{need_mb:.0f} MB, "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py:81:                f"only ~{free_mb:.0f} MB free. Free some
space and retry."
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:93:        "vram_used_mb": used,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\utils\run_telemetry.py:94:        "vram_free_mb": free,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\reporting\harvest.py:40:            meta["size_mb"] =
round(os.path.getsize(video_path) / (1024 * 1024), 1)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\reporting\spec.yaml:42:  # The provider REFUSES clips over its upload cap
(providers/gemini.py::MAX_UPLOAD_MB)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
Cqr7zJeJ.js:60:[Omitted long matching line]
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py:528:                output_size_mb =
os.path.getsize(output_path) / (1024 * 1024)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\stitcher\compiler.py:530:                    f"Dynamic highlight reel
saved: {output_path} ({output_size_mb:.1f} MB)"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:257:                chunk_size_mb =
os.path.getsize(chunk_path) / (1024 * 1024)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:259:                chunk_size_mb = 0.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:268:            metrics["file_size_mb"] =
chunk_size_mb
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:374:            sz = f"{m.get('file_size_mb',
0):.1f}MB" if m else "N/A"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:661:        # 4K chunks (~1.3GB/90s) exceed the
provider's MAX_UPLOAD_MB and are refused, so
```

12. user (2026-07-04T11:49:47.550Z)

```
"""Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING M8/M9, #473).

The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in a
session scratchpad ? this commits the generator so the deploy is reproducible from a clean
clone. It emits a Knative service manifest for::

    python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
    gcloud run services replace service.yaml --region asia-southeast1

Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container command is
a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows the gcloud
cmd-shim mangles those through `--command/--args`, while a YAML round-trips them verbatim. Run
```

```
gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).

The app is configured through a config.json baked INTO the manifest: the wrapper base64-decodes
it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before exec'ing the
server, so one image serves every config shape (config.json wins over the cloud-serving preset ?
backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9) before the
service faces testers again.

Deliberate manifest choices (learned on the A1 deploy):
  * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job worker drains
    on a background thread; without always-allocated CPU a 202'd compile stalls (README §3a).
  * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a second
    instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise it only
    once state is shared.
  * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is rejected
    with it; the code default ("rally-worker") is already correct.
  * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers the
    committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
    `skip_ai_handoff=true` and never needs it).
"""

from __future__ import annotations

import argparse
import base64
import json
import sys

DEFAULT_PROJECT = "gen-lang-client-0306026826"
DEFAULT_REGION = "asia-southeast1"
DEFAULT_SERVICE = "rally-control-plane"
DEFAULT_SERVING_BUCKET = "gs://khelsutra-rally-serving"
DEFAULT_CF_TEAM_DOMAIN = "https://khelsutra.cloudflareaccess.com"
DEFAULT_CF_ACCESS_AUD = "c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"


def build_config(args: argparse.Namespace) -> dict:
    """The app config the wrapper materializes at RALLY_APP_HOME/config.json."""
    bucket = args.serving_bucket.rstrip("/")
    return {
        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
        # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this service.
        "indexing": {"skip_ai_handoff": True},
        "storage": {
            "backend": "gcs",
            "output_root": bucket,
            "input_backend": "gcs",
            "input_root": f"{bucket}/videos",
        },
        "security": {
            "edge_auth_enabled": args.edge_auth == "on",
            "cf_team_domain": args.cf_team_domain,
```

```
            "cf_access_aud": args.cf_access_aud,
            "allowed_video_root": "/tmp/apphome/output",
        },
    }


def build_yaml(args: argparse.Namespace) -> str:
    """Render the Knative service manifest for `gcloud run services replace`."""
    cfg_b64 = base64.b64encode(
        json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
    ).decode("ascii")
    # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so the
    # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
    wrapper = (
        "mkdir -p /tmp/apphome && "
        f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
        "export RALLY_APP_HOME=/tmp/apphome && "
        "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
    )
    return f"""\
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: {args.service}
  labels:
    cloud.googleapis.com/location: {args.region}
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "{args.min_instances}"
        autoscaling.knative.dev/maxScale: "{args.max_instances}"
        run.googleapis.com/cpu-throttling: "false"
    spec:
      timeoutSeconds: {args.timeout}
      containers:
        - image: {args.image}
          command: ["/bin/bash"]
          args: ["-c", "{wrapper}"]
          ports:
            - containerPort: 8080
          resources:
            limits:
              cpu: "{args.cpu}"
              memory: {args.memory}
          env:
            - name: DEPLOYMENT_PROFILE
              value: cloud-serving
            - name: CLOUD_RUN_REGION
              value: {args.region}
            - name: GOOGLE_CLOUD_PROJECT
              value: {args.project}
            - name: RALLY_ALLOWED_HOSTS
              value: "*"
"""


def main(argv: "list[str] | None" = None) -> int:
    p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    p.add_argument(
        "--edge-auth",
        choices=("on", "off"),
        required=True,
        help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
```

```
"
            "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
        )
        p.add_argument("--project", default=DEFAULT_PROJECT)
        p.add_argument("--region", default=DEFAULT_REGION)
        p.add_argument("--service", default=DEFAULT_SERVICE)
        p.add_argument(
            "--image",
            default=None,
            help="Container image; default = the project's rally-worker:latest (post-#470 it "
            "carries cloud-run + auth, so the worker image IS the control-plane image).",
        )
        p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
        p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
        p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
        p.add_argument("--min-instances", type=int, default=1)
        p.add_argument(
            "--max-instances",
            type=int,
            default=1,
            help="Default 1: per-instance SQLite state splits across instances (issue #471).",
        )
        p.add_argument("--cpu", type=int, default=2)
        p.add_argument("--memory", default="4Gi")
        p.add_argument("--timeout", type=int, default=300)
        p.add_argument("-o", "--output", default="-", help="Output path, or '-' for stdout.")
        args = p.parse_args(argv)
        if args.image is None:
            args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-worker:latest"

        text = build_yaml(args)
        if args.output == "-":
            sys.stdout.write(text)
        else:
            with open(args.output, "w", encoding="utf-8", newline="\n") as f:
                f.write(text)
        return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

13. user (2026-07-04T11:49:48.842Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:419:
``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:422:
if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:25:
min_resolution_width: int = 1280
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:261:
ai_chunk_scale_width: int = 1280
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\reporting\harvest.py:233:    # used its internal fallback (30fps / 1280px) ?
surface that as the footgun it is,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\reporting\harvest.py:239:                30.0  # matches the runners' fallback
(_probe_fps_width -> 30.0 / 1280)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
```

```
indexer\backend\worker\__main__.py:376:    cv2's ``_probe_fps_width`` silently defaults to (30,
1280) on a read miss ? fine for the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:719:        default=1280.0,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\reporting\spec.yaml:55:      indexing.ai_chunk_scale_width at its default (1280)
so AI chunks are
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\__init__.py:6:from
backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\golden_fixtures.py:254:    fw, fh = 1280.0, 720.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\gemini_refine.py:82:    clip_scale_width: Optional[int] = 1280,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\gemini_refine.py:136:    clip_scale_width: Optional[int] = 1280,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\gemini_refine.py:221:    clip_scale_width: Optional[int] = 1280,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\gemini_refine.py:332:        default=1280,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py:37:            "resolution_fps_check:
fallback ffprobe timed out after %s",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py:48:        return "resolution_fps_check"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py:62:                "resolution_fps_check:
ffprobe_meta not in shared context ? "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\validators\resolution_fps.py:63:                "re-probing. Ensure
format_check runs before resolution_fps_check "
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\gemini.py:337:                f"Set
indexing.ai_chunk_scale_width (default 1280) so chunks are re-encoded/"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\segmenters\trajectory_hybrid.py:120:        return (fps if fps > 0 else
30.0, width if width > 0 else 1280.0)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\person_detector.py:213:            frame_width = 1280.0  #
Sensible fallback
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\stub_runner.py:44:    frame_width: float = 1280.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\stub_runner.py:68:
frame_width=float(s.get("frame_width", 1280.0)),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\stub_runner.py:154:    frame_width: float = 1280.0,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py:369:            frame_width = 1280.0
```

14. user (2026-07-04T11:50:03.203Z)

```
class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    pts_max_gap_seconds: float = Field(
```

```
        default=10.0,
        ge=0.0,
        description=(
            "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
            "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
            "a gap >10s suggests edited/missing content. Mobile phones, screen "
            "recordings, and other encoders may use different GOP sizes ? tune "
```

15. user (2026-07-04T11:50:04.105Z)

```
    verbose_http: bool = False


class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]

    # --- M9 edge auth (Cloudflare Access, D9) ---
    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-Assertion`
    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the identity
    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF Access
```

16. user (2026-07-04T11:50:13.094Z)

```
        "video_id": job.get("video_id"),
        "error": job.get("error"),
        "result": result,
        "reports": (result or {}).get("reports"),
        "created_at": job.get("created_at"),
        "started_at": job.get("started_at"),
        "finished_at": job.get("finished_at"),
    }


# --- Chunked upload (B2, PR-2) -------------------------------------------------------
# The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives in
# backend.api.uploads as a self-contained APIRouter. It resolves the live startup config via
# backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
# sequential-part logic, and abort cleanup ? is unchanged.
from backend.api import uploads as uploads_api

app.include_router(uploads_api.router)


```

```
# --- Highlight download (B3, PR-2) ---------------------------------------------------


@app.get("/api/highlights/{video_id}/{filename}")
def download_highlight(video_id: str, filename: str, request: Request):
    """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
    (which only writes into output_dir; the browser previously got back an unreachable
    server-local path)."""
    try:
        name = safe_output_filename(filename)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    if not request.app.state.db.get_video(video_id):
        raise HTTPException(status_code=404, detail="Indexed video record not found.")
    # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed URL (bytes flow
    # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local /tmp). The
    # exists/sign logic (with the active storage config) lives in orchestration.signed_reel_url to keep
    # this module lean; best-effort ? any error falls back to the local file below.
    try:
        signed = signed_reel_url(request.app.state.config, video_id, name)
        if signed:
            return RedirectResponse(signed, status_code=307)
    except Exception as e:  # noqa: BLE001 ? signing/exists hiccup ? serve the local file instead
        logger.warning("[highlights] signed-url path failed for %s; serving local: %s", video_id, e)
    output_dir = (
        getattr(
            getattr(request.app.state.config, "output", None), "output_dir", "output"
        )
        or "output"
    )
    path = os.path.join(output_dir, name)
    try:
        path = resolve_under_root(path, output_dir)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    if not os.path.isfile(path):
        raise HTTPException(
            status_code=404,
            detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
        )
    media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
    return FileResponse(path, media_type=media, filename=name)


@app.post("/api/query")
def query_play_segments(req: QueryRequest, request: Request):
    filters = {
        "server": req.server,
        "receiver": req.receiver,
        "duration_min": req.duration_min,
        "event_type": req.event_type,
    }
    segments = request.app.state.db.query_play_segments(req.video_id, filters)
    return {"play_segments": segments}


@app.post("/api/compile", status_code=202)
def compile_highlights(req: CompileRequest, request: Request):
    """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
```

```
reel (#329).
1099
```

17. user (2026-07-04T11:50:14.637Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\main.py:997:@app.get("/api/videos/{video_id}/cost")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\main.py:1043:@app.get("/api/highlights/{video_id}/{filename}")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:406:
# => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:343:#: Video extensions accepted under <corpus>/videos/
(mirrors the labels/ '.csv' filter so a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:504:        for vuri in
storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\worker\__main__.py:522:                    f"[worker] no video for {name!r}
under {corpus}/videos/ ? skipping."
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\static\index.html:35:                        <input type="text" id="video-
path-input" class="text-input" placeholder="e.g. c:/videos/badminton_gopro.mp4" value="">
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\static\app.js:420:
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\ref.py:84:
(``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py:751:
mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py:755:
return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\orchestration.py:860:
download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-
Cqr7zJeJ.js:60:[Omitted long matching line]
```

18. user (2026-07-04T11:50:22.100Z)

```
262:        # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
263:        # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged URI
below is
264:        # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
265:        # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left at
its
266:        # default, while a bare/local-looking root can't stage no matter what input_backend
says
267:        # (resolve_input_uri only consults input_backend when input_root is EMPTY, and an
empty root
268:        # gives us nothing to build the staged URI under).
269:        input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
272:                storage.parse_uri(input_root).backend if input_root else "local"
274:        except ValueError:  # unsupported scheme in input_root ? nowhere real to stage to
278:                "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
279:                "stage local uploads into; a path local to this machine can't be read by the
Cloud "
284:        staged_uri = f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
286:            notify("staging upload to bucket")
291:                staged_uri,
296:        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
```

```
not a 500
298:                "[API] cloud-serving: failed to stage %s -> %s: %s",
300:                staged_uri,
305:                f"cloud-serving: failed to stage the uploaded file to {input_root}: {e}",
308:        # Repoint BOTH the dispatch and the DB video record at the staged bucket object: the
L4 job
311:        req.video_path = staged_uri
312:        input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
314:            video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
424:    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
445:    notify = progress or (lambda stage: None)
787:    notify = progress or (lambda stage: None)
```

19. user (2026-07-04T11:50:30.429Z)

```
cloudflared.config.example.yml
DEPLOY_ALPHA.md
droplet
GPU_VM_SERVING.md
stage_from_bucket.sh
ANNOTATE_AND_TRAIN.md
archives
DRY_RUN_CUJ.md
LOCAL_APPLIANCE_ARCHITECTURE.md
MULTILINGUAL_EXPANSION.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
STUDIO_MEDIA_LIFECYCLE.md
```

20. user (2026-07-04T11:50:31.287Z)

```
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\NEXT_STEPS.md:8:D1 **and emails a summary to
the owner on every registration** (Email Routing send binding ? verified
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\NEXT_STEPS.md:67:  Node host also emails
(currently `noopNotify`). Not urgent ? Cloudflare is the single canonical host.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md:11:This turns the local-
only flow (run engine + SPA on your box, drive it yourself) into a **page you can share with
alpha testers** ? they open `app.khelsutra.guru`, sign in (email allowlist), point at a video,
and get a reel. No SSH, no home IP exposed, no inbound ports.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md:24: Cloudflare edge
(free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH hostnames
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md:81:[Omitted long
matching line]
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\DEPLOY_ALPHA.md:113:3. **Policy:**
Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25
testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\README.md:74:### Let your OWN crawler
through (allowlist; don't weaken protection globally)
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\REQUEST_ACCESS_FORM.md:84:| P2 | Worker +
D1 + `wrangler.toml` + `schema.sql` + **unit tests (10 green)** | ? | ? | **LIVE on Cloudflare**
(Worker `hidden-king-khelsutra` + D1 `khelsutra-access`; persists + emails per submission) |
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\engine.env.example:17:RALLY_ALLO
WED_HOSTS=api.khelsutra.guru,localhost,127.0.0.1   # REQUIRED ? load-bearing fix (allowlist is
decoupled from the bind); loopback added for on-box health checks
C:\Users\avidu\Projects\khelsutra-
guru\khelsutra\deploy\droplet\README.md:15:`api.khelsutra.guru`**, team domain
**`https://khelsutra.cloudflareaccess.com`**, allowlist
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\deploy\droplet\README.md:89:   - Policy:
*Allow* ? *Emails* ? **`avi.dullu@gmail.com`** (one-time-PIN; add tester emails later).
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\src\deploy-config.test.js:38:  // The
Email Routing send binding is what emails the owner on each registration (PR #8).
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\src\assets.test.js:485:// be an
allowlisted literal: brand, email/URL/flyer-autonym link, storage product name). This is the
```

C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\src\assets.test.js:492: // and the bare separators/symbols. Links to mailto:/http(s):/flyer (emails, URLs, autonyms) are skipped.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\src\assets.test.js:505:   // "wordy" English = at least one Latin word of 2+ letters that isn't an allowlisted token.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\docs\archives\FRIENDLY_AND_MULTILINGUAL.md:68:- **CI enforcement (keeps it "by design"):** (1) **key-parity** ? every locale has exactly the `en` key set (drop a stub `ta.json` ? CI lists every gap = the extensibility proof); (2) **no-bare-strings** ? flags user-facing literals not routed through `t()` (curated allowlist to start).
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\site\public\robots.txt:9:# allowlist the crawler in Cloudflare (see site/README.md ? "Crawling / bot protection").

21. user (2026-07-04T11:50:39.214Z)

~~~~
# Deploy KhelSutra Studio for alpha ? a shareable page, no SSH

> **Status:** `RETIRED / HISTORICAL` (updated 2026-07-02). The alpha this runbook stood up went live on the DO droplet `168.144.126.176` on 2026-06-22 (`deploy/droplet/README.md`), and that droplet was **decommissioned + deleted 2026-06-25** (PR [#117](https://github.com/avidullu/khelsutra/pull/117), commit `5ffb750`; its data archived to `gs://khelsutra-rally-corpus/archive/droplet-claude-do-rally-test-20260625/`). Alpha serving is **paused pending the Cloud-Run move**. The Cloudflare/engine mechanics below remain the reference for the next stand-up, but every "droplet" reference is historical ? do **not** deploy to or SSH that IP (DigitalOcean can recycle it).
>
> **Durable topology decision ? single-origin is the DEFAULT for SPA + CF-Access.** The **split** topology this runbook originally shipped (SPA on Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Access over both) broke Cloudflare Access in practice ? the cross-origin split defeated the Access session/preflight flow and locked testers out (issues [#97](https://github.com/avidullu/khelsutra/issues/97), [#114](https://github.com/avidullu/khelsutra/issues/114)); the 2026-06-22 fix repointed serving to a **single origin** (engine-served SPA). Any future stand-up should serve the SPA and the API from ONE origin (`docs/LOCAL_APPLIANCE_ARCHITECTURE.md` D1 / the §0 "Alternative (single-origin)" note below). This paragraph records the decision so #97 can be closed.
>
> **Still pending (owner, Cloudflare dashboard):** clean up the dangling `app.`/`api.khelsutra.guru` CNAMEs and the CF-Access application ? they still resolve via Cloudflare to a tunnel with no origin.
>
> *As-shipped facts (historical): **Topology:** split (superseded ? see above) · **Cost:** ~$0/mo standing (Pages + Tunnel + Access free ?50 users) + Gemini per-run (~$0.05/match).*

This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a **page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.

> **Need more compute?** To run the engine on a **GCP L4 GPU VM** (more CPU/RAM for ffmpeg
> + optional WASB GPU detection) behind the same tunnel + Access, see **[`GPU_VM_SERVING.md`](GPU_VM_SERVING.md)**
> ? no DNS change (~$500?600/mo while the VM runs). *Its "droplet kept as fallback" language is
> historical too ? there is no droplet anymore.*

## 0. The shape

```
 tester browser
    ? https
    ?
 Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH hostnames
    ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA; built with VITE_API_BASE)
~~~~

    ??? api.khelsutra.guru ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on YOUR box
                                                                                ? FastAPI engine
                                                                                ? (Gemini segmenter; GPU optional)
                                                                                ? async jobs, SQLite, output/ on disk
```

- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`. The box's home/public IP is never exposed.
- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to the tunnel can't spoof identity.
- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-driven).

> **Single-origin is now the DEFAULT (decision recorded 2026-07-02 ? see the status banner / issue #97):** run **one box** that serves the SPA *and* the API behind one origin (engine-served SPA, or a Caddy reverse-proxy ? `docs/LOCAL_APPLIANCE_ARCHITECTURE.md` D1), so Cloudflare Access sees a single origin. The **split** topology this runbook originally shipped (stable Pages-hosted page + a thin API tunnel) is kept below for reference but is **superseded** ? it broke the Access sign-in flow (#97, #114).

---

1. Prerequisites

- A **box** to run the engine: any Linux VM (Gemini-only is fine for alpha ? the retired stand-up used a CPU DO droplet, deleted 2026-06-25), a **GCP L4 GPU VM** (`GPU_VM_SERVING.md`), or your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.10+ installed; the `badminton-highlight-indexer` engine checked out.
- A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing site there).
- `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/downloads/`).
- A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks to Google; this is a paid + cloud action).

---

2. Engine (the `api.` side)

2.1 Install (with the auth extra)
```bash
cd badminton-highlight-indexer
pip install -e .[auth]          # PyJWT[crypto] ? REQUIRED when edge auth is on, else every authenticated request is rejected 401
```

2.2 The safe-exposed config
Create / edit `config.json` on the box. **Where it must live:** the engine reads `$RALLY_APP_HOME/config.json` if `RALLY_APP_HOME` is set, else `./config.json` relative to *the directory you launch the server from* ? **not** necessarily the repo root. If the engine reads a different (or absent) file it silently falls back to all-defaults, which means **edge auth is OFF**. To make writer and reader agree by construction, write it with `python -m backend.main --setup` (it targets the exact path the server reads) and edit *that* file:
```jsonc
{
  "security": {
    "edge_auth_enabled": true,                         // turn the CF-Access JWT verify ON
    "allowed_video_root": "/srv/khelsutra/incoming",  // the path-jail ? REQUIRED when exposed

```
    "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER allowed_video_root (see §7)
    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
    "cf_access_aud": "<the CF Access application AUD>" // filled in §4
  },
  "indexing": {
    "default_segmenter": "gemini" // ? CPU box MUST set this ? the shipped default `yolo_hybrid` needs torch/GPU, and the SPA sends no segmenter, so every job fails "No module named 'torch'" without it
  }
}
```
And export the env (a `.env` or your shell):
```bash
export GEMINI_API_KEY="<your-rotated-key>"
export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA origin
export RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1"  # REQUIRED ? see ? below; without it EVERY tunneled request 400s (loopback added so on-box /api/health checks still pass ? setting this var REPLACES the loopback-only default)
export RALLY_BIND_HOST="127.0.0.1"                             # keep the loopback bind (cloudflared dials 127.0.0.1:8000)
```

> **? Two env vars you MUST set, or the tunnel is dead-on-arrival.** With `edge_auth_enabled: true`, the engine (a) flips its bind to `0.0.0.0` and (b) keeps the DNS-rebinding **Host-header guard** at its loopback-only default ? so it boots "healthy" but then **`400`s every request `cloudflared` forwards with `Host: api.khelsutra.guru`** (including the §6 `/api/health` check). The boot only *warns* about this (`EXPOSED_HOST_ALLOWLIST_LOOPBACK`), it does not stop. Set **`RALLY_ALLOWED_HOSTS="api.khelsutra.guru,localhost,127.0.0.1"`** (the load-bearing fix ? the Host allowlist is independent of the bind; the loopback entries keep the on-box `/api/health` check passing, since setting this var REPLACES the loopback-only default) and **`RALLY_BIND_HOST="127.0.0.1"`** (so the bind stays loopback as §0/§6 claim; `cloudflared` dials loopback anyway). *Verified against engine `origin/master`: with the allowlist unset, `GET /api/health` with `Host: api.khelsutra.guru` ? `400`; with it set ? `200`.*

> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.

2.3 Run
```bash
python -m backend.main --server     # binds 127.0.0.1:8000 ? kept loopback by RALLY_BIND_HOST (§2.2); with edge_auth ON and that var unset it would bind 0.0.0.0 instead
```
Keep it alive with a process manager (`systemd` on a Linux VM, mirroring `site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

---

3. The SPA on Cloudflare Pages (the `app.` side)

The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-deploy on merge to `master`).

1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick the `khelsutra` repo.
2. **Build settings:**
   - **Root directory:** `web`
   - **Build command:** `npm run build`
   - **Build output directory:** `web/dist`
   - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this is what points the SPA at the engine tunnel (PR #64).
3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

> Every push to `master` now rebuilds + redeploys the SPA. To preview a change first, Pages builds a per-PR preview URL.

---

4. Cloudflare Access (gate BOTH hostnames)

1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

---

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                  # browser auth to your zone
cloudflared tunnel create khelsutra-studio                # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNEL_UUID>
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
cloudflared tunnel run khelsutra-studio                   # or install as a service: `cloudflared service install`
```
See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).

---

6. Verify (the posture test + the CUJ)

Exposure posture (must hold ? this is what makes it safe to share):
- From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?

The shareable CUJ (what a tester does): open `app.khelsutra.guru` ? sign in ? **upload a video from their device** (the SPA chunked-upload lands it in `upload_dir`, inside the jail) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page. *(Pasting a `gs://?` URI does NOT work on this box ? see §7: the `allowed_video_root` jail rejects every non-local URI. Browser upload is the alpha path.)*

---

7. Honest caveats / deferred

- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).

146   - **`gs://` / `s3://` / `gdrive://` URIs are REJECTED on the exposed box.** Setting `allowed_video_root` (required to expose, §2.2) path-jails `/api/process` to *local* paths under that root; the engine resolves a `gs://?` URI to a local path that escapes the jail ? `400` (pinned by engine `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`). Cloud-URI ingest works **only** when `allowed_video_root` is unset (the local single-user posture ? the opposite of this runbook). For alpha, the ingest path is **browser upload** (it lands inside the jail). *(Scheme-exempting recognized buckets from the jail would be an engine change, not a config flip.)*
147   - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
148   - **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
149   - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ~95 MB chunks (the SPA chunked-upload path honours the engine's `max_part_mb`, default **95**; lower `security.max_part_mb` to shrink them). `gs://` bucket pre-staging is **not** an option on this box (see the jail caveat above).
150   - **Box offline = engine offline.** The Pages SPA stays up and degrades gracefully ? today that means the SetupWizard's offline branch (driven by its `/api/capabilities` fetch), **not** a persistent "engine offline" banner; the SPA has no `/api/health` ping (unimplemented ? corrected 2026-07-02). Processing resumes when the box is back. Beta removes this with a queue (`HOSTING_PLAN.md` §3).
151   - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for clips you're OK sending.
152
153   ---
154
155   ## 8. Teardown
156   `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages custom domain; disable the Access application; stop the engine service. The box's `output/` + `uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
157
158   ---
159   ### Changelog
160   - 2026-07-02 ? **RETIRED / HISTORICAL.** The droplet this runbook deployed to was decommissioned + deleted **2026-06-25** (PR #117; data archived to `gs://khelsutra-rally-corpus/archive/droplet-claude-do-rally-test-20260625/`) ? no `.md` recorded that until now. Recorded the durable **single-origin-default** topology decision for SPA + CF-Access (#97, #114); re-scoped the §1 box prerequisite to "any Linux VM"; softened the unimplemented `/api/health` "engine offline banner" claim (§7); flagged the CF-dashboard CNAME/Access cleanup as pending owner click-ops.
161   - 2026-06-22 ? **correctness pass** (adversarial verify against engine `origin/master` + a hermetic repro). Fixed two would-break-the-deploy gaps: (1) **`RALLY_ALLOWED_HOSTS` was missing** ? with `edge_auth` on, the Host-header guard 400s every tunneled request until it's set (added it + `RALLY_BIND_HOST` to §2.2; repro confirmed 400?200); (2) **`gs://` ingest is rejected** by the `allowed_video_root` jail on the exposed box (dropped the `gs://` promise from §6/§7; alpha path = browser upload). Minor: 401-not-500 wording (§2.1), `config.json` location/`--setup` note (§2.2), bind comment (§2.3), ~95 MB chunk size (§7).
162   - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative) + the engine `HOSTING_PLAN.md` §2.
163

22. user (2026-07-04T11:50:50.802Z)

DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
README.md
RUNLOG_truly-local_TEMPLATE.md

23. user (2026-07-04T11:50:51.899Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md:81:  `cf_access_aud=c980da67?082b` (the existing "KhelSutra Studio (alpha)" CF Access app already lists
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md:82:  `api.khelsutra.guru` in `self_hosted_domains` ? no new app needed; same 4-email allowlist policy),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md:93:stale CNAME to a dead CF tunnel. To make the alpha reachable-with-auth by allowlisted testers: (a) add
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md:96:service (Cloud Run domain-mapping in asia-southeast1, or a CF origin-rule Host override); (c) re-point the
```

24. user (2026-07-04T11:50:56.591Z)

```
# A1 alpha serving deploy ? hosted CUJ run report (cloud-serving)

**Date:** 2026-07-04 · **Region:** asia-southeast1 · **Project:** gen-lang-client-0306026826
**Scope:** A1 (ALPHA_LAUNCH_READINESS §7) ? stand up the alpha serving endpoint and verify the
hosted `process ? jobs ? compile ? download` CUJ against the WASB-only owned detector.

> Companion to `DEPLOY_REPORT_cloud-serving_2026-07-04.md` (PR #457, the GPU-batching runlog). That
> report proved the L4 worker + measured the batching win via the M6 dispatch **shim**; THIS report
> proves the deployed **control-plane service** end-to-end. They do not overlap.

## 1. What was deployed

| Resource | Value |
|---|---|
| L4 worker **Job** | `rally-worker` ? nvidia-l4, 8 vCPU / 32Gi, `--no-gpu-zonal-redundancy`, `--max-retries 0 --task-timeout 3600`, GCS-FUSE mount `khelsutra-rally-serving`?`/gcs`, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar` |
| Control-plane **Service** | `rally-control-plane` ? rev 00002, `--no-cpu-throttling`, `min/maxScale 1/3`, cpu 2 / mem 4Gi, port 8080, **private (IAM-gated, no allUsers)** |
| Service image | `asia-southeast1-docker.pkg.dev/?/rally/rally-control-plane:latest` (= `rally-worker:latest` + `google-cloud-run`; see §5 bug 1) |
| Service URL | `https://rally-control-plane-2bosgeukpa-as.a.run.app` |
| Config (via base64 startup wrapper ? `RALLY_APP_HOME=/tmp/apphome/config.json`) | `deployment.profile=cloud-serving`, `compute_target=cloud_run`, `indexing.skip_ai_handoff=true`, `storage.backend=gcs`, `output_root=gs://khelsutra-rally-serving`, `input_backend=gcs`, `input_root=?/videos` |
| Env | `DEPLOYMENT_PROFILE=cloud-serving`, `CLOUD_RUN_REGION=asia-southeast1`, `GOOGLE_CLOUD_PROJECT=?`, `RALLY_ALLOWED_HOSTS=*` (CLOUD_RUN_JOB is reserved ? code default `rally-worker` is correct) |

**Boot-crash trap avoided:** the `cloud-serving` preset forces `indexing.skip_ai_handoff=False`
(Gemini ON), which would trip the `AI_HANDOFF_KEY_MISSING` startup check (no Gemini key) and
crash-loop. Verified `loader.py:194` deep-merges `defaults < preset < config.json`, so the
explicit `skip_ai_handoff=true` wins. Also pinned `storage.backend=gcs` explicitly to satisfy the
`STORAGE_INCOHERENT` check. Boot log confirmed: `deployment profile ACTIVE: cloud-serving
(compute_target=cloud_run)` + `Uvicorn running on http://0.0.0.0:8080`.

```

2. Health / capabilities (authenticated via `gcloud auth print-identity-token`)

- `GET /api/health` ? `200 {"status":"ok","sport":"badminton","default_segmenter":"yolo_hybrid"}`
- `GET /api/capabilities` ? `segmenters` incl. `wasb_hybrid`; `deployment.cloud_available=true`; `gemini_key_present=false` (no crash ? skip_ai_handoff honored); `gcs` backend usable.

3. gs:// bucket-path CUJ ? PROVEN end-to-end

1. `POST /api/process {"video_path":"gs://khelsutra-rally-serving/videos/MahadevpuraSingles.MP4","segmenter":"wasb_hybrid","compute":"cloud"}` ? `202 {job_id: d16654d1?}`
2. Poll `GET /api/jobs/{id}`: `validating ? dispatching (cloud_run) ? done`.
   - CP pulled + validated the video (format/resolution/pts/scene/blur/relevance all pass).
   - Dispatched L4 Job execution `rally-worker-k49kg`; worker log: `WASB in FULLY-LOCAL mode (skip_ai_handoff=True) ? no gemini handoff, no API key needed`, `torch 1.11.0+cu113 cuda True`.
   - **Result: `success` ? "Successfully indexed 48 play segments (cloud_run / 'wasb_hybrid')"**; outputs at `gs://khelsutra-rally-serving/outputs/915473?/`.
   - Wall: created 02:12:42 ? finished 02:26:27 (**~13m45s**, incl. L4 cold image pull + inference).
3. `POST /api/compile {video_id, segment_ids:[1..48], output_filename:"mahadevpura_wasb_highlights.mp4"}` ? `202 {job_id: d947a9e6?}` ? poll ? **`success`**. In-process ffmpeg stitch (trim 48 clips w/ padding ? concat ? watermark) on the CP; **`Dynamic highlight reel saved ? (166.0 MB)`**; wall ~3m26s (02:27:47?02:31:13, incl. re-fetch of the gs:// source).
4. `GET /api/highlights/{video_id}/mahadevpura_wasb_highlights.mp4` ? the reel is served (correct `206`/`video/mp4`/etag/size headers), and the **full 174,038,655-byte reel was retrieved via <32 MiB Range requests and ffprobes as a VALID video: H.264 1920×1080 @ 29.97fps + AAC, duration 244.3s** (48 rallies). ? **A single full `GET` returns edge-500** ? Cloud Run's ~32 MiB response cap vs. the `FileResponse` fixed `Content-Length` (see §5 bug 3).

4. Notes / observations

- **In-process compile serializes behind process jobs** (drain `ThreadPoolExecutor(max_workers=1)`): a queued compile waits while a process job holds the worker in its bucket-poll. Fine for the alpha (low concurrency); worth noting for load.
- **WASB batch tuning is NOT forwarded on the dispatch path.** `_maybe_dispatch_remote` forwards only `skip_ai_handoff` (orchestration.py:274-283); the L4 job runs its baseline batch (not the preset's batch64/prefetch4), so the measured #457 ?30.5% win is not applied via the deployed CP. Minor (correctness unaffected); follow-up = forward `indexing.wasb.*` overrides like `skip_ai_handoff`.
- `player_pool` / `max_frames` are dropped on the cloud path (already noted at orchestration.py:273).

5. Bugs surfaced + handled

1. **`google-cloud-run` missing from the image** (BLOCKER). First real CP deploy failed every dispatch with *"google-cloud-run is required for CloudRunCompute's real client"* ? the image installed only `.[storage-gcs]`; the reused-as-control-plane role needs `google.cloud.run_v2`. **Unblocked** with a derived image (`rally-worker` + `pip install google-cloud-run`). **Durable fix: PR #462** (adds a `cloud-run` extra + `.[storage-gcs,cloud-run]` in the Dockerfile + test guard; 19

tests pass).

2. **Browser-upload ? gs:// staging gap.** Uploads land on the CP's local FS (`/api/upload/complete` ? `/tmp/apphome/output/uploads/?`), and `_maybe_dispatch_remote` hard-rejects local input under cloud-serving (orchestration.py:250-255). Upload endpoints verified working; the gap is staging. **Filed: issue #461** (fix = stage local?gs://input_root before dispatch).
3. **`/api/highlights` download 500s for reels >32 MiB.** `FileResponse` sets a fixed `Content-Length`; Cloud Run caps a single buffered HTTP/1 response at ~32 MiB (empirically: 29.6 MB Range ? 206, 32.4 MB ? 500). App logs 200; Google Frontend returns 500. Every real reel exceeds 32 MiB. **Filed: issue #463** (fix = serve reels via a gs:// signed-URL redirect; also fixes the ephemeral `/tmp` reel location). Workaround verified: ranged download reassembles a byte-exact, valid reel.

6. M9 edge auth ? ENABLED + gate verified (DNS routing remains, owner-handoff)

Cloudflare Access edge-auth is **ON** (rev `rally-control-plane-00004-tvr`) and enforcing:
- Image rebuilt to add `PyJWT[crypto]` (the `auth` extra) + the `google-cloud-run` client.
- Config: `security.edge_auth_enabled=true`, `cf_team_domain=https://khelsutra.cloudflareaccess.com`, `cf_access_aud=c980da67?082b` (the existing "KhelSutra Studio (alpha)" CF Access app already lists `api.khelsutra.guru` in `self_hosted_domains` ? no new app needed; same 4-email allowlist policy), `allowed_video_root=/tmp/apphome/output`.
- **Bug found + fixed to make M9 compatible with cloud-serving:** enabling edge-auth requires `allowed_video_root`, and its jail (`validate_input_path`?`resolve_under_root`) `realpath()`-mangled and **rejected gs:// inputs** ? so edge-auth would have 400'd the proven gs:// CUJ. **PR #465** makes the local-path jail skip storage URIs. Overlaid into the deployed image (build-time asserted); runtime-verified (Phase 1): gs:// ? 202 accepted; a local out-of-root path ? 400 (jail still enforced).
- **Verified (Phase 2):** `/api/health` (exempt) ? 200; `/api/process` without a CF JWT ? **401** ("missing Cf-Access-Jwt-Assertion header"). Service boots (no crash-loop from the exposure checks).

Remaining (owner, CF/GCP dashboard ? outward-facing production DNS): `api.khelsutra.guru` is still a stale CNAME to a dead CF tunnel. To make the alpha reachable-with-auth by allowlisted testers: (a) add `allUsers`?`roles/run.invoker` on the Cloud Run service (so CF's proxied requests reach the app ? the app still requires the CF JWT, so direct run.app access stays 401); (b) route `api.khelsutra.guru` to the service (Cloud Run domain-mapping in asia-southeast1, or a CF origin-rule Host override); (c) re-point the CF DNS record (proxied) off the dead tunnel. Then the authed CUJ works end-to-end through the CF login.
Full end-to-end authed CUJ verification needs (a)+(b) (a CF-issued JWT only arrives via the CF edge).

Rollback (make it owner-usable again without CF): redeploy with `EDGE_AUTH` unset (edge-auth off) via `scratchpad/gen_service_yaml.py` ? `gcloud run services replace`.

```

## 7. Cost ledger + teardown

- One L4 execution (~14 min, 8 vCPU + 1×L4) for the CUJ. CP service holds `min-instances 1` (small
  always-on CPU cost). Owner directive: **keep running** (closed alpha).
- Teardown (if needed): `gcloud run services delete rally-control-plane`, `gcloud run jobs delete
  rally-worker`, both `--region asia-southeast1`; bucket/secret/repo retained.

```

25. user (2026-07-04T11:51:34.581Z)

```
No matches found
```

26. user (2026-07-04T11:51:40.276Z)

```
No matches found
```

27. user (2026-07-04T11:51:47.933Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\cloudrun\README.md:13:- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore ? see
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:38:| **GPU VM** | **NOT created** ? blocked on `GPUS_ALL_REGIONS` quota (see §2) | **~$0.35/hr (T4) running, ~$0.01/hr stopped** | `teardown.sh` (stop or delete) |
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:59:## 2. ? GPU quota ? the one remaining owner action
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:61:New GCP projects start at **`GPUS_ALL_REGIONS` = 0**, the *global* cap that gates GPU creation even
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:62:when a regional per-type quota is non-zero. Confirmed 2026-06-20:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:66:GLOBAL       GPUS_ALL_REGIONS = 0  ? BLOCKS the VM
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:69:**Owner:** IAM & Admin ? **Quotas** ? filter **"GPUs (all regions)"** ? Edit ? request **1** ? submit.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:74:
--flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep GPUS_ALL_REGIONS
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:79:> **? Spot does NOT skip the quota.** TESTED 2026-06-20: a **Spot** T4 create also fails with
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:80:> `Quota 'GPUS_ALL_REGIONS' exceeded. Limit: 0.0 globally` ? `GPUS_ALL_REGIONS` gates **both**
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:82:> was filed via the Cloud Quotas API: `POST ?/quotaPreferences` `GPUS-ALL-REGIONS-per-project`
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:85:> once the quota is granted, and the resumable cache makes preemption recoverable.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:89:## 3. Create the GPU VM (after quota ? 1)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\deploy\gcp\README.md:117:**Spot variant (cheaper; still needs the §2 quota ? Spot is NOT a quota bypass):** same command with
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:40:Also confirm: **L4 GPU quota** in `$REGION`; the **WASB weights** at
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\CLOUD_GPU_DRYRUN_GUIDE.md:31:3. **GPU quota** ? `GPUS_ALL_REGIONS` must be ? 1 (already granted). Check:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\CLOUD_GPU_DRYRUN_GUIDE.md:33:   gcloud compute project-info describe
--flatten="quotas[]" \
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\CLOUD_GPU_DRYRUN_GUIDE.md:34:     --format="table(quotas.metric,quotas.limit)" |
```

```
grep GPUS_ALL_REGIONS
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\DATA_IN_GCS.md:253:4.
**Gating infra (owner-only, not done):** GPU `GPUS_ALL_REGIONS` quota is still **0** (no VM ever
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\archives\decisions\DECISIONS.md:538:- **New-project GPU gate:** `GPUS_ALL_REGIONS`
quota = 0 must be raised to ?1 (owner console
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\archives\decisions\DECISIONS.md:588:  (native==WSL 1194/1195; deterministic) ? see
PR #259. **Spot does NOT bypass `GPUS_ALL_REGIONS`**
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\archives\decisions\DECISIONS.md:589:  (tested) ? both on-demand + Spot need the
quota; a `GPUS_ALL_REGIONS?1` increase was filed via the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\archives\decisions\DECISIONS.md:590:  Cloud Quotas API (pending). GCP GPU quota
matters for the GCP *serving* spike, not the DO training run.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\archives\decisions\DECISIONS.md:675:- **GCP works today:** quota granted
(`GPUS_ALL_REGIONS=1`), bucket + SA + `$100` budget provisioned, and M1/M2/M3.5 ran on an L4 for
~$1?2/run with no subscription. Real money, but trivial per run ? and **one** portable
PyTorch+CUDA stack (local ? GPU VM ? Cloud Run), no second provider to maintain.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\HOSTING_PLAN.md:109:limit + per-user upload quota ? `security.allowed_video_root`
set ? upload size cap + extension
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\SETUP_NEW_MACHINE.md:223:Drive files locally ? it needs no service account, no API
quota, no OAuth, and Drive streams + caches
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\STORAGE_SHARING_MODEL.md:27:| Tier | What | Size reality | Stored? | Quota? |
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\STORAGE_SHARING_MODEL.md:32:Why this split: 2 GB cannot hold even one raw match,
but holds **~25?40 reels** ? the quota is
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\STORAGE_SHARING_MODEL.md:48:- **Grace flow:** over quota ? banner + email with the
exact list the chosen policy would evict ?
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\docs\STORAGE_SHARING_MODEL.md:56:   ? compile reel(s) ? reel lands in LIBRARY (quota
meter ticks)

[Showing results with pagination = limit: 30]
```

28. user (2026-07-04T11:51:49.284Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\api\job_worker.py:45:#   - Cloud Run poll has a max_wait_sec bound
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\compute\cloud_run.py:10:     cold-start is absorbed by the async poll. On a poll
TIMEOUT (``max_wait_sec`` can undercut the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\compute\cloud_run.py:245:        The poll budget (``max_wait_sec``, default
1800s) can undercut the deployed job's own
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:169:            poll_every_n_sec=10.0, max_wait_sec=1200.0
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:186:                f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up."
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\providers\gemini.py:190:                f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:8:  ``max_wait_sec`` deadline).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:42:    """An op hung past ``op_timeout_sec``
or never became ready within ``max_wait_sec``."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\infrastructure\execution_policy.py:54:    max_wait_sec: float = 1800.0  # total
```

```
wall budget for a poll loop
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\execution_policy.py:162:    ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is still in progress.
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\execution_policy.py:173:        policy.max_wait_sec,
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\execution_policy.py:183:        if elapsed >= policy.max_wait_sec:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\infrastructure\execution_policy.py:188:                f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
```

29. user (2026-07-04T11:53:04.351Z)

```
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md:4:> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/past_projects/` when DONE)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md:99:2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md:102:This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.
```

30. user (2026-07-04T11:53:05.065Z)

```
        )
    return video, kept, out_name


def reel_object_uri(cfg, video_id: str, name: str) -> str:
    """The bucket URI a compiled reel is mirrored to for a direct signed-URL download (#463), or "" when
    the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the compile
    mirror-upload and the /api/highlights redirect: ``<output_root>/highlights/<video_id>/<name>``."""
    root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
    if not root or "://" not in root:
        return ""
    return f"{root.rstrip('/')}/highlights/{video_id}/{name}"


def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
    """The active storage settings dict (per-backend config for the storage layer ? S3 endpoint/region,
    GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-default backend
    isn't silently constructed as a fresh default."""
    sc = getattr(cfg, "storage", None)
    return sc.model_dump() if sc is not None else None
```

31. assistant (2026-07-04T11:53:37.893Z)

All evidence gathered. Compiling the section now.

32. user (2026-07-04T11:55:05.963Z)

Structured output provided successfully